

# **LAPORAN TUGAS BESAR**

## **MATA KULIAH IF2211 STRATEGI ALGORITMA**

**Pemanfaatan Algoritma Greedy Dalam Pembuatan Bot di Game Robocode: Tank  
Royale**

Dosen Pengampu:

**Monterico Adrian, S.T., M.T.**



Disusun oleh :

|                            |          |
|----------------------------|----------|
| Rafa Abdussalam Danadyaksa | 13523133 |
| Lukas Raja Agripa          | 13523158 |
| Muhammad Rizain Firdaus    | 13523164 |

**SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA**

**INSTITUT TEKNOLOGI BANDUNG**

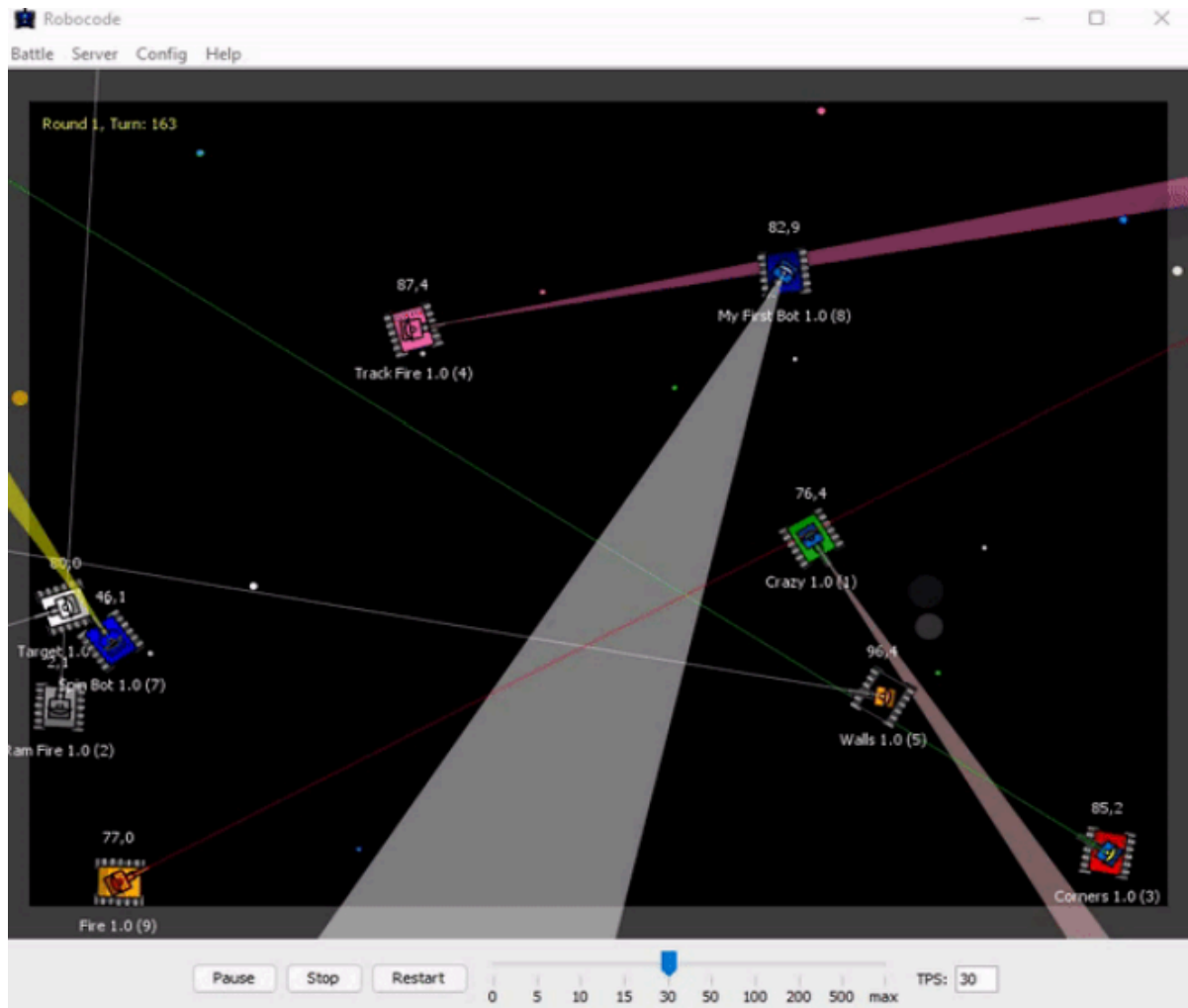
**2025**

# DAFTAR ISI

|                                                         |           |
|---------------------------------------------------------|-----------|
| <b>DAFTAR ISI.....</b>                                  | <b>2</b>  |
| <b>Bab 1: Deskripsi Tugas.....</b>                      | <b>5</b>  |
| <b>Bab 2: Landasan Teori.....</b>                       | <b>12</b> |
| 2.1 Algoritma Greedy.....                               | 12        |
| 2.2 Bot dan Game Engine.....                            | 13        |
| 2.2.1 Pengimplementasian Algoritma Greedy pada Bot..... | 13        |
| 2.2.2 Menjalankan Bot pada Permainan.....               | 14        |
| 2.3 Struktur Direktori Program.....                     | 15        |
| <b>Bab 3: Aplikasi Strategi Greedy.....</b>             | <b>17</b> |
| 3.1. Mapping Strategi Greedy Pada Bot.....              | 17        |
| 3.2. Alternatif Solusi.....                             | 17        |
| 3.2.1. Alternatif Bot Ackerman.....                     | 17        |
| 3.2.2. Alternatif Bot Reiss.....                        | 18        |
| 3.2.3. Alternatif Bot Reiner.....                       | 18        |
| 3.2.4. Alternatif Bot Yeager.....                       | 19        |
| 3.3. Analisis Efektifitas dan Efisiensi Solusi.....     | 22        |
| 3.4. Strategi Greedy Terpilih.....                      | 24        |
| <b>Bab 4: Implementasi dan Pengujian.....</b>           | <b>27</b> |
| 4.1 Implementasi.....                                   | 27        |
| 4.1.1. Ackerman (Bot Utama).....                        | 27        |
| 4.1.2. Reiss.....                                       | 31        |
| 4.1.3. Reiner.....                                      | 34        |
| 4.1.4. Yeager.....                                      | 36        |
| 4.2 Penjelasan Struktur Data, Fungsi, dan Prosedur..... | 39        |

|                                                       |           |
|-------------------------------------------------------|-----------|
| 4.2.1 Bot Ackerman.....                               | 39        |
| 4.2.1.1. Atribut Kelas Ackerman.....                  | 39        |
| 4.2.1.1. Fungsi dan Prosedur Utama Ackerman.....      | 40        |
| 4.2.1.1. Fungsi dan Prosedur Alternatif Ackerman..... | 43        |
| 4.2.2 Bot Reiss.....                                  | 46        |
| 4.2.2.1. Atribut Kelas Reiss.....                     | 46        |
| 4.2.2.2. Metode Utama Reiss.....                      | 46        |
| 4.2.2.3. Metode Alternatif Reiss.....                 | 48        |
| 4.2.3 Bot Reiner.....                                 | 48        |
| 4.2.3.2 Metode Utama Reiner.....                      | 49        |
| 4.2.3.3 Metode Alternatif Reiner.....                 | 52        |
| 4.3.3 Bot Yeager.....                                 | 53        |
| 4.3.3.1. Atribut Kelas Yeager.....                    | 53        |
| 4.3.3.2. Metode Utama Yeager.....                     | 54        |
| 4.3.4.1 Metode Alternatif Yeager.....                 | 55        |
| 4.3. Pengujian.....                                   | 58        |
| 4.3.1. Hasil Pengujian.....                           | 58        |
| 4.3.2. Analisis Pengujian.....                        | 58        |
| <b>Bab 5: Kesimpulan dan Saran.....</b>               | <b>60</b> |
| 5.1. Kesimpulan.....                                  | 60        |
| 5.2. Saran.....                                       | 60        |
| Lampiran.....                                         | 62        |
| Daftar Pustaka.....                                   | 63        |

## Bab 1: Deskripsi Tugas



*Gambar 1 Robocode Tank Royale*

Robocode adalah permainan pemrograman yang bertujuan untuk membuat kode bot dalam bentuk tank virtual untuk berkompetisi melawan bot lain di arena. Pertempuran Robocode berlangsung hingga bot-bot bertarung hanya tersisa satu seperti permainan Battle Royale, karena itulah permainan ini dinamakan Tank Royale. Nama Robocode adalah singkatan dari "Robot code," yang berasal dari versi asli/pertama permainan ini.

Robocode Tank Royale adalah evolusi/versi berikutnya dari permainan ini, di mana bot dapat berpartisipasi melalui Internet/jaringan. Dalam permainan ini, pemain berperan sebagai programmer bot dan tidak memiliki kendali langsung atas permainan. Pemain hanya bertugas untuk membuat program yang menentukan logika atau "otak" bot. Program yang dibuat akan berisi instruksi tentang cara bot bergerak, mendeteksi bot lawan, menembakkan senjatanya, serta bagaimana bot bereaksi terhadap berbagai kejadian selama pertempuran.

Pada Tugas Besar pertama Strategi Algoritma ini, mahasiswa diminta untuk membuat sebuah bot yang nantinya akan dipertandingkan satu sama lain. Tentunya mahasiswa harus menggunakan strategi greedy dalam membuat bot ini. Komponen-komponen dari permainan ini antara lain:

## **1. Rounds dan Turns**

Pertempuran dapat terdiri dari beberapa rounds. Secara default, satu pertempuran berisi 10 rounds, di mana setiap rounds akan memiliki pemenang dan yang kalah. Setiap round dibagi menjadi beberapa turns, yang merupakan unit waktu terkecil. Satu turn adalah satu ketukan waktu dan satu putaran permainan. Jumlah turn dalam satu round tergantung pada berapa lama waktu yang dibutuhkan hingga hanya tersisa bot terakhir yang bertahan. Pada setiap turn, sebuah bot dapat:

- Menggerakkan bot, memindai musuh, dan menembakkan senjata.
- Bereaksi terhadap peristiwa seperti saat bot terkena peluru atau bertabrakan dengan bot lain atau dinding.
- Perintah untuk bergerak, berputar, memindai, menembak, dan sebagainya dikirim ke server untuk setiap turn.

Perlu diperhatikan bahwa API (Application Programming Interface) bot resmi secara otomatis mengirimkan niat bot ke server di balik layar, sehingga Anda tidak perlu mengkhawatirkannya, kecuali jika Anda membuat API Bot sendiri. Pada setiap turn, bot

akan secara otomatis menerima informasi terbaru tentang posisinya dan orientasinya di medan perang. Bot juga akan mendapatkan informasi tentang bot musuh ketika mereka terdeteksi oleh pemindai. Perlu diketahui bahwa game engine yang akan digunakan pada tugas besar ini tidak mengikuti aturan default mengenai komponen Round & Turns.

## **2. Batas Waktu Giliran**

Penting untuk dicatat bahwa setiap bot memiliki batas waktu untuk setiap turn yang disebut turn timeout, biasanya antara 30-50 ms (dapat diatur sebagai aturan pertempuran). Ini berarti bahwa bot tidak bisa mengambil waktu sebanyak yang mereka inginkan untuk bergerak dan menyelesaikan turn saat ini. Setiap kali turn baru dimulai, penghitung waktu ulang diatur ulang dan mulai berjalan. Jika batas waktu tercapai dan bot tidak mengirimkan pergerakannya untuk turn tersebut, maka tidak ada perintah yang dikirim ke server. Akibatnya, bot akan melewatkan turn tersebut. Jika bot melewatkan turn, ia tidak akan bisa menyesuaikan gerakannya atau menembakkan senjatanya karena server tidak menerima perintah tepat waktu sebelum turn berikutnya dimulai.

## **3. Energi**

Semua bot memulai permainan dengan jumlah energi awal sebanyak 100 poin energi.

- Bot akan kehilangan energi jika ditembak atau ditabrak oleh bot musuh.
- Bot juga akan kehilangan energi jika menembakkan meriamnya.
- Bot akan mendapatkan energi jika peluru dari meriamnya mengenai musuh. Energi yang didapat akan lebih banyak 3 kali lipat dari energi yang digunakan untuk menembakkan peluru.
- Bot dengan energi nol akan dinonaktifkan dan tidak bisa bergerak. Jika bot terkena serangan dalam keadaan ini, bot akan hancur.

## **4. Peluru**

Semakin banyak energi (daya tembak) yang digunakan untuk menembakkan peluru, semakin berat peluru tersebut dan semakin lambat gerakannya. Namun, peluru yang lebih berat juga menghasilkan lebih banyak kerusakan dan memungkinkan bot mendapatkan lebih banyak energi saat mengenai bot musuh. Seperti disebutkan sebelumnya, peluru yang lebih berat akan bergerak lebih lambat. Ini berarti akan membutuhkan waktu lebih lama untuk mencapai target, meningkatkan risiko peluru tidak mengenai sasaran. Sebaliknya, peluru yang lebih ringan bergerak lebih cepat, sehingga lebih mudah mengenai target, tetapi peluru ringan tidak memberikan banyak poin energi saat mengenai bot musuh.

## **5. Panas Meriam (Gun Heat)**

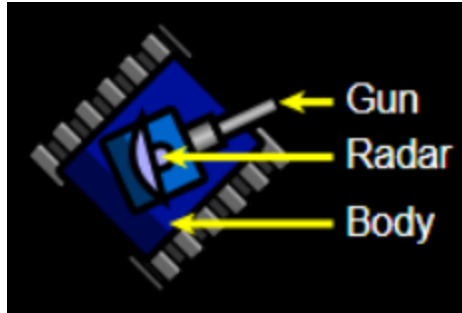
Saat menembakkan peluru, meriam akan menjadi panas. Peluru yang lebih berat menghasilkan lebih banyak panas dibandingkan peluru yang lebih ringan. Ketika meriam terlalu panas, bot tidak dapat menembak hingga suhu meriam turun ke nol. Selain itu, meriam juga sudah dalam keadaan panas di awal round dan perlu waktu untuk mendingin sebelum bisa digunakan untuk pertama kalinya.

## **6. Tabrakan**

Perlu diperhatikan bahwa bot akan menerima kerusakan jika menabrak dinding (batas arena), yang disebut wall damage. Hal yang sama juga terjadi jika bot bertabrakan dengan bot lain. Jika bot menabrak bot musuh dengan bergerak maju, ini disebut ramming (menabrak dengan sengaja), yang akan memberikan sedikit skor tambahan bagi bot yang menyerang.

## **7. Bagian Tubuh Tank**

Tubuh tank terdiri dari 3 bagian:



- Body adalah bagian utama dari tank yang digunakan untuk menggerakkan tank.
- Gun digunakan untuk menembakkan peluru dan dapat berputar bersama body atau independen dari body.
- Radar digunakan untuk memindai posisi musuh dan dapat berputar bersama body atau independen dari body.

## 8. Pergerakan

Bot dapat bergerak maju dan mundur hingga kecepatan maksimum. Dibutuhkan beberapa giliran untuk mencapai kecepatan maksimum. Bot dapat mengalami percepatan maksimum sebesar 1 unit per giliran dan pengereman dengan perlambatan maksimum 2 unit per giliran. Percepatan dan perlambatan maksimum tidak tergantung pada kecepatan bot saat itu.

## 9. Berbelok

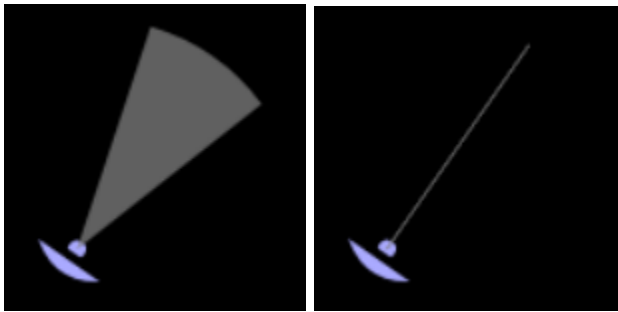
Seperti yang disebutkan sebelumnya, bagian tubuh, turret (meriam), dan radar dapat berputar secara independen satu sama lain. Jika turret atau radar tidak diputar, maka keduanya akan mengarah ke arah yang sama dengan tubuh bot. Setiap bagian tubuh memiliki kecepatan putar yang berbeda. Radar adalah bagian tercepat dan dapat berputar hingga 45 derajat per giliran, yang berarti dapat berputar 360 derajat dalam 8 giliran. Turret dan meriam dapat berputar hingga 20 derajat per giliran. Bagian paling lambat adalah tubuh tank, yang dalam kondisi terbaik dapat berputar hingga 10 derajat per giliran. Namun, ini bergantung pada kecepatan bot saat ini. Semakin cepat bot bergerak,



semakin lambat kemampuannya untuk berbelok. Perlu diperhatikan bahwa tidak ada energi yang dikonsumsi saat bot bergerak atau berbelok.

## 10. Pemindaian

Aspek penting dalam Robocode adalah memindai bot musuh menggunakan radar. Radar dapat mendeteksi bot dalam jangkauan hingga 1200 piksel. Musuh yang berada lebih dari 1200 piksel dari bot tidak dapat terdeteksi atau dipindai oleh radar. Penting untuk diperhatikan bahwa sebuah bot hanya dapat memindai bot musuh yang berada dalam jangkauan sudut pemindaian (scan arc)-nya. Sudut pemindaian ini merupakan "sapuan radar" dari arah radar sebelumnya ke arah radar saat ini dalam satu giliran.



Jika radar tidak bergerak dalam suatu giliran, artinya radar tetap mengarah ke arah yang sama seperti pada giliran sebelumnya, maka sudut pemindaian akan menjadi nol derajat, dan bot tidak akan dapat mendeteksi musuh. Oleh karena itu, sangat disarankan untuk selalu mengubah arah radar agar tetap dapat memindai musuh.

## 11. Skor

Pada akhir pertempuran, setiap bot akan di ranking berdasarkan total skor yang diperoleh masing-masing bot selama keseluruhan pertempuran. Tentunya, tujuan utama pada tugas besar ini adalah membuat bot yang memberikan skor setinggi mungkin. Berikut adalah rincian komponen skor pada pertempuran:

- **Bullet Damage:** Bot mendapatkan **poin sebesar damage** yang dibuat kepada bot musuh menggunakan peluru.

- **Bullet Damage Bonus:** Apabila peluru berhasil membunuh bot musuh, bot mendapatkan **poin sebesar 20% dari damage** yang dibuat kepada musuh yang terbunuh.
- **Survival Score:** Setiap ada bot yang mati, bot lainnya yang masih bertahan pada ronde tersebut mendapatkan **50 poin**.
- **Last Survival Bonus:** Bot terakhir yang bertahan pada suatu ronde akan mendapatkan **10 poin dikali dengan banyaknya musuh**.
- **Ram Damage:** Bot mendapatkan **poin sebesar 2 kalinya damage** yang dibuat kepada bot musuh dengan cara menabrak
- **Ram Damage Bonus:** Apabila musuh terbunuh dengan cara ditabrak, bot mendapatkan **poin sebesar 30% dari damage** yang dibuat kepada musuh yang terbunuh.

Skor akhir bot adalah akumulasi dari 6 komponen diatas. Perlu diperhatikan bahwa game akan menampilkan berapa kali suatu bot meraih peringkat 1, 2, atau 3 pada setiap ronde. Namun, hal ini tidak dihitung sebagai komponen skor maupun untuk perangkingan akhir. Bot yang dianggap menang pertempuran adalah bot dengan akumulasi skor tertinggi. Untuk informasi lebih lengkap, silahkan buka dokumentasi Tank Royale pada link [berikut](#).

## Bab 2: Landasan Teori

### 2.1 Algoritma Greedy

Algoritma greedy adalah algoritma yang memecahkan persoalan secara langkah per langkah (*step by step*) sedemikian sehingga, pada setiap langkah:

1. Mengambil pilihan yang terbaik yang dapat diperoleh pada saat itu tanpa memperhatikan konsekuensi ke depan (prinsip “*take what you can get now!*”).
2. Dan “berharap” bahwa dengan memilih optimum lokal pada setiap langkah akan berakhir dengan optimum global.

Elemen-elemen dari Greedy sendiri didapatkan sebagai berikut:

1. Himpunan kandidat (**C**) : berisi kandidat yang akan dipilih pada setiap Langkah (misal: simpul/sisi di dalam graf, job, task, koin, benda, karakter, dsb).
2. Himpunan solusi (**S**) : berisi kandidat yang sudah dipilih.
3. Fungsi solusi: menentukan apakah himpunan kandidat yang dipilih sudah memberikan solusi.
4. Fungsi seleksi (**selection function**): memilih kandidat berdasarkan strategi greedy tertentu. Strategi greedy ini bersifat heuristik.
5. Fungsi kelayakan (**feasible**): memeriksa apakah kandidat yang dipilih dapat dimasukkan ke dalam himpunan solusi (layak atau tidak).
6. Fungsi obyektif : memaksimumkan atau meminimumkan.

Algoritma greedy melibatkan pencarian sebuah himpunan bagian,  $S$ , dari himpunan kandidat,  $C$ ; yang dalam hal ini,  $S$  harus memenuhi beberapa kriteria yang ditentukan, yaitu  $S$  menyatakan suatu solusi dan  $S$  di optimisasi oleh fungsi objektif.

Akan tetapi, solusi optimum global yang dihasilkan algoritma greedy belum tentu merupakan solusi terbaik karena sangat mungkin solusi tersebut merupakan solusi sub-optimum atau pseudo-optimum. Ada dua alasan kenapa hal tersebut bisa terjadi, yaitu sebagai berikut:

1. Algoritma greedy tidak beroperasi secara menyeluruh terhadap semua kemungkinan solusi yang ada (sebagaimana pada metode exhaustive search).
2. Terdapat beberapa fungsi seleksi yang berbeda, sehingga perlu memilih fungsi yang tepat jika algoritma diinginkan untuk menghasilkan solusi optimal.

## **2.2 Bot dan Game Engine**

### **2.2.1 Pengimplementasian Algoritma Greedy pada Bot**

Dalam permainan Robocode, pemain dinyatakan sebagai programmer dikarenakan pemain harus mengimplementasikan bot atau tank mereka dengan memprogram bot yang ingin dibuat. Namun, di sisi lain permainan Robocode ini memberikan sebuah dokumentasi API yang dapat dijadikan sebagai referensi untuk dapat membuat bot mereka. Akses API ini dilakukan dengan menggunakan atribut, metode, atau fungsi yang sudah disediakan oleh game engine. Referensi API ini memungkinkan untuk para programmer (pemain) agar dapat mengembangkan bot mereka sesuai keinginan dan referensi masing-masing.

Dengan ini pemain dapat mengimplementasikan algoritma dan strategi menurut preferensi masing-masing pemain. Maka dari itu, pengimplementasian algoritma greedy pada pembuatan bot sangat memungkinkan untuk dapat memenangkan bot ketika bertarung dengan bot lainnya sehingga mencapai tujuan akhir dari pembuatan bot itu sendiri yakni untuk dapat memenangkan permainan. Greedy yang diterapkan pada pembuatan bot ini menekankan langkah atau pilihan optimal dari masing-masing bot untuk dapat memenangkan pertandingan, dengan memperhatikan atribut atau *state* dari sang bot sebagai pilihan dalam peninjauan pengambilan keputusan. Tidak hanya itu, programmer (pemain) dapat juga meninjau dari *event* yang terjadi dan melakukan *internal source code* dan *conditional statements* yang mendukung optimalisasi pengambilan keputusan yang berujung pada strategi greedy dari sang bot tersebut.

### 2.2.2 Menjalankan Bot pada Permainan

Seusai programmer (pemain) membuat bot mereka, programmer (pemain) tidak perlu *hard-code* untuk menjalankan program mereka pada *game engine* yang sudah ada. Namun, programmer (pemain) wajib untuk menyesuaikan kode mereka dengan pembacaan pada program *game engine*, selebihnya dapat dilihat pada link dokumentasi [berikut](#). Dipastikan juga, pemain memenuhi file-file yang diperlukan agar ketika bot dijalankan pada game engine dapat berjalan dengan baik. Selain itu, mendaftarkan bot pada game engine juga berbeda dengan memperlihatkan bot kita pada program hingga terbaca. Untuk mendaftarkan bot yang sudah dibuat dapat diperhatikan langkah-langkah sebagai berikut:

1. Pastikan bot sudah disimpan dalam sebuah folder yang berisikan file dengan tipe *.cmd*, *.cs*, *.csproj*, *.json*, dan *.sh*. Adapun isi dari masing-masing file dapat dilihat dari dokumentasi contoh bot yang telah dibuat oleh developer pada link [berikut](#).
2. Simpan folder yang berisikan file-file tersebut lalu, apabila terdapat beberapa folder bot, simpan folder-folder tersebut dalam satu folder

direktori sehingga ketika melakukan proses *booting*, bot-bot dapat langsung terdaftar semua tanpa perlu mengubah direktorinya satu per satu.

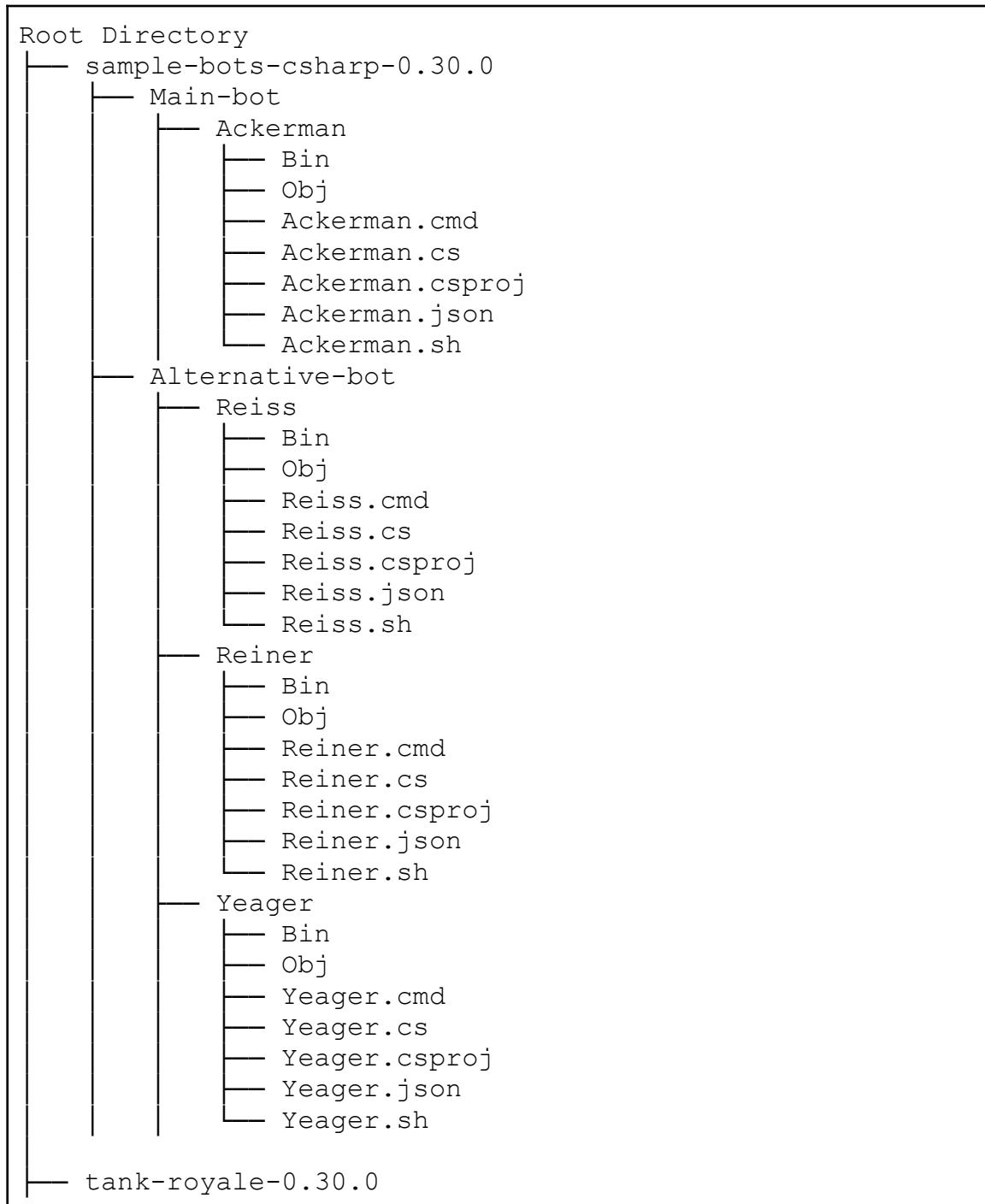
3. Pastikan masing-masing folder bot telah di-compile, jika belum gunakan perintah `dotnet build` pada direktori, lalu `dotnet run` untuk menghubungkannya ke localhost.
4. Lalu jalankan game engine dengan menjalankan executable file java sesuai direktori yang sudah ditentukan (Executable Jar akan dibuat pada directory `gui-app/build/libs`. Jar dapat dijalankan dengan command berikut:  

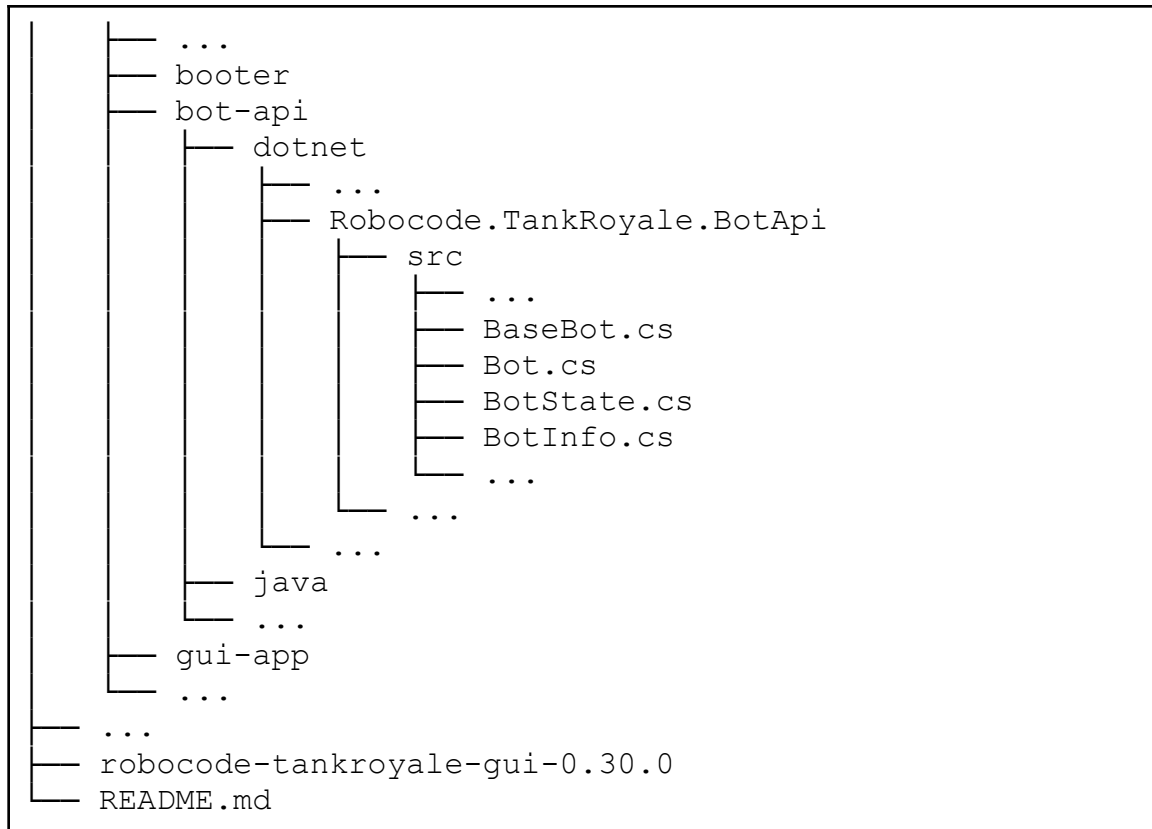
```
java -jar  
./gui-app/build/libs/robocode-tankroyale-gui-0.30  
.0.jar).
```
5. Selanjutnya lakukan proses booting dengan memilih direktori tempat folder yang berisikan bot-bot yang ingin dimainkan dengan klik tombol “Config” lalu klik tombol “Bot Root Directories” dan klik tombol “add”, lalu tambahkan direktori.
6. Jalankan permainan dengan klik tombol “Start Battle”, lalu akan muncul panel permainan. Pilihlah bot yang ingin anda daftarkan dalam permainan, jika sudah lakukan proses booting, tambahkan bot dengan klik tombol “Add All”, lalu jalankan permainan dengan klik tombol “Start Battle”.

Bot akan berjalan dan melakukan aksinya sesuai dengan hasil kode yang sudah dibuat oleh programmer (pemain). Dalam hal ini aksi yang dilakukan oleh bot dihasilkan dari referensi API yang digunakan dan kondisi lainnya yang mungkin ditambahkan oleh programmer (pemain).

## 2.3 Struktur Direktori Program

Program permainan dan bot yang dibuat dalam tugas besar 1 memiliki struktur direktori sebagai berikut:





Proses pembuatan algoritma greedy pada bot hanya akan dilakukan pada folder /sample-bots-csharp-0.30.0>NamaBot. File lain di luar folder sample-bots-csharp-0.30.0>NamaBot berfungsi sebagai komponen untuk berkomunikasi dengan game engine dan untuk menjalankan bot.



## Bab 3: Aplikasi Strategi Greedy

### 3.1. Mapping Strategi Greedy Pada Bot

Pada permainan Robocode, setiap bot memiliki tujuan yang sama yakni untuk mengalahkan semua bot musuh dan memenangkan pertandingan. Adapun tujuan ini dilakukan dengan cara memaksimalkan total score di akhir permainan. Hal ini dapat dicapai dengan menggunakan sebuah algoritma greedy yang diimplementasikan pada masing-masing bot yang kami buat. Sebelumnya pembagian masing-masing elemen dari greedy pada bot nantinya dapat dilihat sebagai berikut:

#### a. Himpunan Kandidat

Himpunan kandidat yang kami pilih yakni menyesuaikan dengan tujuan dan jalan untuk memenangi permainan yang dilihat dari total score yang ada. Maka dari itu, himpunan kandidat yang mencakup yakni dimulai dari atribut bot, pergerakan bot, penembakan bot, *scanning*, pergerakan senjata pada bot, pergerakan radar/scanner pada bot, dan pengambilan keputusan lainnya berdasarkan segala tahapan dari bot untuk menghasilkan solusi yakni dalam memenangkan permainan.

#### b. Himpunan Solusi

Sedangkan himpunan solusi yang ada untuk masing-masing bot dipilih untuk dapat mencapai tujuan yakni dimulai dari kombinasi pemilihan himpunan kandidat seperti kombinasi pergerakan pada bot yang dimodifikasi tiap bot, kombinasi penyerangan dari segi kekuatan tembakan, optimalisasi kekuatan penembakan berdasarkan parameter, eksekusi aksi yang dilakukan ketika melakukan scanning pada bot musuh, dan penyesuaian pada atribut bot.

#### c. Fungsi Solusi

Fungsi solusi pada implementasi bot yang dikembangkan yakni dengan menentukan apakah suatu rangkaian sudah dapat dilakukan, seperti implementasi pemilihan strategi bot dengan dibagi sebuah fase berdasarkan atribut energi bot, pemantauan bot musuh dengan menggunakan sebuah atribut bot baik musuh ataupun sebaliknya, dan lainnya.

d. Fungsi Seleksi

Fungsi seleksi memilih sebuah kandidat terbaik dengan melakukan perbandingan pada kandidat yang lainnya, seperti pemilihan target (bot musuh) berdasarkan jarak yakni dipilih yang paling dekat ataupun berdasarkan energi yang paling rendah, pemilihan metode penembakan berdasarkan energi bot dan jarak terhadap bot musuh, dan pemilihan gerakan yang optimal untuk menghindari musuh.

e. Fungsi Kelayakan

Fungsi kelayakan yakni untuk memastikan langkah atau pengambilan keputusan dari sang bot itu didasari pada sebuah syarat yang jelas dan tidak lewat dari suatu batasan yang ada (valid). Seperti misalnya melalui pembatasan energi (apabila energi terlalu rendah maka bot tidak akan melakukan penembakan) atau pembatasan jarak (bot tidak dapat menembak jika bot musuh terlalu jauh).

f. Fungsi Objektif

Fungsi objektif yakni untuk memaksimalkan dan mengoptimalkan tindakan oleh bot. Hal ini didasarkan oleh solusi yang sudah melewati tahap seleksi dan kelayakan pada semestinya, seperti memaksimalkan kerusakan dengan melakukan penyerangan secara berturut apabila kondisi bot memenuhi, meminimalkan kerusakan diri dengan menghindari penyerangan dari bot musuh dapat dilakukan dengan mengoptimalkan

pergerakan dan menempatkan posisi yang aman untuk dirinya, atau dengan memaksimalkan potensi peluang untuk bertahan diri hingga memenangkan permainan. Hal tersebutlah yang dapat membuat strategi ini menjadi strategi yang layak dan menggaris besarkan greedy pada masing-masing bot.

## 3.2. Alternatif Solusi

### 3.2.1. Alternatif Bot Ackerman

Bot Ackerman menerapkan algoritma greedy dalam strategi pertempurannya untuk menyerang musuh secara efektif dan mengoptimalkan pergerakan di arena. Strategi ini berfokus pada pengambilan keputusan yang memberikan keuntungan maksimal pada setiap langkah, dengan tujuan mengurangi energi musuh secepat mungkin sambil tetap menjaga ketahanan bot.

Dalam implementasi algoritma greedy, bot ini menggunakan elemen-elemen berikut:

#### a. Himpunan Kandidat

Kandidat yang tersedia bagi bot untuk dipilih pada setiap langkah meliputi berbagai aspek permainan, seperti posisi bot sendiri, posisi musuh, energi yang tersisa, arah pergerakan, dan sudut tembakan yang mungkin dilakukan. Informasi ini terus diperbarui berdasarkan pemindaian terhadap kondisi di arena.

#### b. Himpunan Solusi

Himpunan solusi terdiri dari keputusan-keputusan yang telah diambil oleh bot selama permainan, seperti apakah ia akan menembak, bergerak ke sudut arena, atau melakukan penghindaran. Setiap keputusan ini dievaluasi berdasarkan

efektivitasnya dalam mendekati tujuan utama, yaitu mengalahkan musuh sambil tetap bertahan.

c. Fungsi Solusi

Fungsi ini digunakan untuk mengevaluasi apakah kandidat yang dipilih telah membentuk solusi yang efektif. Dalam konteks ini, bot Ackerman menganggap solusi berhasil jika ia dapat menyerang musuh dengan akurasi tinggi tanpa kehilangan banyak energi akibat serangan balik. Jika bot gagal mencapai kondisi ini, ia akan terus menyesuaikan strategi berdasarkan informasi terbaru yang diperoleh dari arena.

d. Fungsi Seleksi

Bot menggunakan fungsi seleksi berbasis heuristik untuk memilih langkah terbaik dalam menyerang atau bertahan. Strategi greedy yang diterapkan meliputi pemilihan target berdasarkan posisi musuh yang paling mudah terkena tembakan, memilih sudut tembakan yang memberikan peluang serangan tertinggi, serta menentukan kapan harus menembak dengan kekuatan penuh atau kapan harus menghemat energi dengan tembakan yang lebih terukur.

e. Fungsi Kelayakan

Sebelum menambahkan suatu keputusan dalam solusi, bot harus memastikan bahwa langkah tersebut layak untuk dilakukan. Sebagai contoh, bot tidak akan menembak jika tidak ada musuh dalam jangkauan yang memungkinkan, dan ia tidak akan bergerak ke arah dinding jika hal tersebut dapat menghambat manuvernya. Fungsi ini memastikan bahwa setiap tindakan yang diambil tidak merugikan bot dalam jangka panjang

#### f. Fungsi Objektif

Tujuan utama bot Ackerman adalah meminimalkan energi musuh secepat mungkin sambil mempertahankan energinya sendiri selama mungkin. Untuk itu, bot akan memaksimalkan jumlah serangan yang akurat dan menghindari serangan musuh sebanyak mungkin. Jika bot menemukan dirinya dalam posisi yang tidak menguntungkan, ia akan segera mencari area yang lebih aman atau mengubah sudut serangannya agar tetap bisa bertarung secara efektif, yaitu dengan *spray*.

#### 3.2.2. Alternatif Bot Reiss

Bot Reiss menerapkan algoritma greedy dalam strategi pertempurannya dengan fokus utama pada efisiensi energi. Strategi ini bertujuan untuk mengoptimalkan serangan terhadap musuh sambil memastikan bot tetap memiliki energi yang cukup untuk bertahan dalam pertempuran. Dengan pendekatan ini, bot dapat menyesuaikan kekuatan serangannya berdasarkan kondisi di arena dan jumlah musuh yang terdeteksi.

Dalam implementasi algoritma greedy, bot ini menggunakan elemen-elemen berikut:

##### a. Himpunan Kandidat

Kandidat yang tersedia bagi bot untuk dipilih pada setiap langkah meliputi berbagai aspek permainan, seperti posisi musuh, jarak antara bot dan musuh, jumlah energi yang tersisa, serta kemungkinan arah pergerakan untuk menghindari atau menyerang

musuh secara efektif. Informasi ini terus diperbarui berdasarkan pemindaian di sekitar arena.

b. Himpunan Solusi

Himpunan solusi terdiri dari serangkaian keputusan yang telah diambil bot selama permainan, termasuk kapan menembak, seberapa besar daya tembak yang digunakan, dan bagaimana bot bereaksi ketika terkena serangan. Setiap keputusan ini dievaluasi berdasarkan efektivitasnya dalam mencapai tujuan utama, yaitu mengalahkan musuh dengan tetap menjaga ketahanan bot.

c. Fungsi Solusi

Fungsi ini digunakan untuk mengevaluasi apakah kumpulan kandidat yang dipilih telah membentuk solusi yang efektif. Dalam konteks ini, bot Reiss menganggap solusi berhasil jika ia dapat menyerang musuh secara efisien tanpa kehilangan terlalu banyak energi. Jika serangan yang dilakukan tidak cukup efektif atau justru menyebabkan bot kehilangan lebih banyak energi dibandingkan musuh, strategi akan disesuaikan dengan kondisi terbaru.

d. Fungsi Seleksi

Bot menggunakan fungsi seleksi berbasis heuristik untuk menentukan langkah terbaik dalam menyerang atau bertahan. Strategi greedy yang diterapkan meliputi pemilihan kekuatan tembakan berdasarkan jarak musuh—di mana bot akan menggunakan daya tembak yang lebih kecil untuk musuh yang jauh dan daya tembak yang lebih besar untuk musuh yang lebih dekat. Selain itu, bot juga menyesuaikan jumlah peluru yang ditembakkan berdasarkan jumlah musuh yang terdeteksi.

e. Fungsi Kelayakan

Sebelum menambahkan suatu keputusan ke dalam solusi, bot harus memastikan bahwa langkah tersebut layak untuk dilakukan. Misalnya, jika energi bot berada di bawah ambang batas tertentu, bot akan mengurangi kekuatan tembakannya untuk menghindari pemborosan energi. Selain itu, bot tidak akan menyerang jika posisi musuh berada di luar jangkauan efektif, dan ia akan lebih memprioritaskan manuver menghindar jika terkena serangan bertubi-tubi.

f. Fungsi Objektif

Tujuan utama bot Reiss adalah menjaga keseimbangan antara menyerang dan mempertahankan energi untuk bertahan lebih lama di arena. Untuk itu, bot akan memaksimalkan efektivitas tembakannya dengan menyesuaikan daya serangan berdasarkan kondisi musuh. Jika jumlah musuh banyak, bot akan menggunakan lebih dari satu peluru dalam satu serangan untuk meningkatkan peluang mengenai target. Sebaliknya, jika musuh yang terdeteksi hanya sedikit, bot akan lebih menghemat energi dengan menembakkan satu peluru per serangan. Selain itu, jika bot terkena tembakan, ia akan segera bergerak ke arah yang berlawanan untuk menghindari serangan lebih lanjut.

### 3.2.3. Alternatif Bot Reiner

Ide dari solusi ini adalah bot akan memiliki fokus utama pada bot akan selalu menyerang musuh dengan energi terendah untuk meningkatkan peluang eliminasi. Bot mengandalkan pemindaian untuk mendeteksi musuh dan menentukan target yang paling menguntungkan berdasarkan energi dan jarak. Saat menemukan musuh, bot menyesuaikan

kekuatan tembakan agar tetap efisien dalam penggunaan energi dan tetap memaksimalkan kerusakan. Tujuan solusi ini adalah meningkatkan efektifitas serangan dengan menargetkan lawan yang lebih mudah dikalahkan, sehingga memperbesar peluang bertahan lebih lama dan mengurangi jumlah ancaman di arena.

a. Himpunan Kandidat

Himpunan kandidat dalam solusi ini akan terdiri dari semua musuh yang terdeteksi di arena. Setiap musuh yang terdeteksi melalui pemindaian radar akan dimasukkan ke dalam daftar target potensial beserta informasi posisinya, jaraknya, dan tingkat energinya.

b. Himpunan Solusi

Himpunan solusi akan berisi daftar target potensial yang paling menguntungkan, yaitu bot yang memiliki energi terendah atau posisi yang terdekat.

c. Fungsi Solusi

Fungsi ini digunakan untuk mengevaluasi apakah kumpulan kandidat yang dipilih telah membentuk solusi yang efektif. Dalam konteks ini, bot reiner menganggap solusi berhasil jika ia dapat menyerang musuh yang memiliki energi terendah dengan pertimbangan jarak musuh terdekat.

d. Fungsi Seleksi

Fungsi Seleksi akan bertanggung jawab untuk menentukan target optimal dari daftar musuh yang tersedia. Seleksi akan melakukan perbandingan energi dari semua musuh yang terdeteksi dan memilih musuh dengan energi terendah sebagai prioritas utama,



dengan jarak sebagai faktor penentu sekunder jika ada musuh dengan energi yang sama.

e. Fungsi Kelayakan

Fungsi Kelayakan akan digunakan untuk memastikan bahwa hanya musuh yang valid dipilih untuk diserang. Jika musuh telah memiliki energi nol atau tidak lagi terdeteksi, mereka akan dihapus dari daftar target untuk menghindari penembakan sia-sia ke arah musuh yang sudah dieliminasi.

f. Fungsi Objektif

Fungsi objektif pada solusi ini adalah mengoptimalkan efektivitas serangan dengan menargetkan lawan yang lebih lemah terlebih dahulu. Dengan strategi ini, bot berusaha mengurangi jumlah lawan secara sistematis untuk meningkatkan peluang bertahan dan memenangkan pertandingan.

### 3.2.4. Alternatif Bot Yeager

Solusi alternatif dari Yeager sendiri yakni untuk dapat memberikan keputusan yang optimal dan konsisten dengan menyesuaikan dengan kondisi dari permainan. Pada dasarnya Yeager juga memiliki tujuan yang sama dengan bot lainnya, yakni untuk dapat mengumpulkan *score* sebanyak mungkin serta juga untuk dapat memenangkan permainan dengan memperhatikan kondisi *energy level* dirinya. Adapun untuk dapat lebih jelas melihat susunan algoritma greedy yang dibuat, perhatikan pembagian komponen-komponen algoritma greedy sebagai berikut:

a. Himpunan Kandidat

Dalam menyelesaikan tujuan akhir dari bot ini yakni untuk memenangkan permainan, Yeager memiliki parameter pengukuran

dari beberapa sisi kandidat yang dapat dipakai untuk menentukan solusi yang ada. Adapun anggota dari himpunan kandidat milik Yeager seperti *energy level* Yeager, ukuran arena, sudut arena, dinding arena, besar jarak dan sudut antara Yeager dengan bot musuh, *gun heat*, bot musuh yang di scan, *firepower*, besaran unit pergerakan, dan arah pergerakan Yeager.

b. Himpunan Solusi

Adapun berdasarkan himpunan kandidat yang dipilih, bot akan melakukan penyesuaian terhadap kondisi yang ada. Pada halnya Yeager akan melakukan pengecekan terhadap status energi miliknya, lalu akan menyesuaikan terhadap tindakan selanjutnya. Jadi, himpunan solusi milik Yeager bertepatan pada banyaknya perubahan fase yang menyesuaikan pada status energi atau *energy level* miliknya. Dan dari fase-fase tersebut dikumpulkan solusi yang dapat mengoptimalkan pengambilan langkah dari Yeager. Seperti contoh kasusnya yakni Yeager akan melakukan penembakan secara maksimal pada saat *energy level*nya tinggi dan mengumpulkan *damage score* dan *bullet damage* sebanyak mungkin hingga akhirnya *total score* yang dihasilkan paling banyak diantara bot lainnya.

c. Fungsi Solusi

Dikarenakan ada tiga fase yang dilakukan pada Yeager, masing-masing fase ini berupa sebuah metode yang dijadikan fungsi untuk dapat memvalidasi langkah dalam pengambilan keputusan sehingga menghasilkan sebuah solusi yang valid. Maka dari itu, fungsi solusi milik Yeager berupa ketiga metode yang mendefinisikan fase-fase pada Yeager (fase menyerang, fase

defensif, dan fase tak-terkendali) yang divalidasi secara langsung melalui fungsi `CheckEnergyAndSwitchPhase()`.

d. Fungsi Seleksi

Fungsi seleksi sendiri merupakan sebuah fungsi untuk dapat memilih langkah yang terbaik dan paling optimal dari sebuah himpunan kandidat. Fungsi ini diterapkan pada proses scanning bot musuh yang terdapat pada metode `OnScannedBot()`. Pada metode ini dilakukan pemosisian senjata sesuai dengan titik koordinat posisi musuh. Serta melakukan penembakan apabila pada jarak tertentu dengan firepower yang disesuaikan terhadap *gun heat* dan jarak Yeager terhadap bot musuh.

e. Fungsi Kelayakan

Fungsi kelayakan sendiri merupakan sebuah fungsi untuk melakukan pengecekan apakah elemen yang dipilih layak dan tidak keluar dari batasan. Fungsi ini diterapkan pada beberapa metode seperti halnya ketika bot ingin melakukan transisi fase, maka dilakukan pengecekan apakah *energy level* sudah memenuhi dan jika iya, maka bot akan masuk ke fase tersebut.

f. Fungsi Objektif

Fungsi objektif yang ada pada Yeager yakni adalah tiap fase yang memiliki implementasinya masing-masing. Metode inilah yang memastikan bahwa Yeager selalu mengambil keputusan yang bersifat terbaik dan optimal sesuai dengan *energy level* Yeager.

### 3.3. Analisis Efektifitas dan Efisiensi Solusi

Berdasarkan algoritma solusi yang telah didapat, dilakukan pengujian untuk menilai penerapan algoritma solusi pada bot dan mencari algoritma solusi terbaik. Pengujian dilakukan dengan proses sebagai berikut.

1. Keempat bot yang sudah dirancang akan dilakukan diadu dalam mode. Efisiensi dan efektifitas dinilai berdasarkan hasil leaderboard setelah permainan berakhir. Pengujian ini dilakukan sebanyak 5 kali.
2. Bot yang memiliki rata-rata poin tertinggi akan bertanding 1 vs 1 melawan bot yang lain. Efisiensi dan efektifitas dinilai berdasarkan hasil leaderboard setelah permainan berakhir. Pengujian ini dilakukan sebanyak 5 kali.

Berikut adalah hasil pengujian dalam mencari algoritma solusi yang terbaik. Algoritma solusi Ackerman, Reiner, Yeager, dan Reiss diadu dengan mode permainan Classic.

*Tabel 3.3.1 Hasil Pengujian 1.*

| Percobaan | Total Perolehan Poin |        |        |       |
|-----------|----------------------|--------|--------|-------|
|           | Ackerman             | Reiner | Yeager | Reiss |
| 1         | 754                  | 680    | 652    | 878   |
| 2         | 1119                 | 841    | 986    | 590   |

|           |       |       |       |       |
|-----------|-------|-------|-------|-------|
| 3         | 888   | 602   | 847   | 628   |
| 4         | 629   | 540   | 602   | 618   |
| 5         | 817   | 839   | 699   | 370   |
| Rata-rata | 841,4 | 700,4 | 757,2 | 616,8 |

Berikut adalah hasil pengujian yang dilakukan terhadap algoritma solusi Ackerman dengan algoritma solusi Yeager dengan tipe permainan 1 vs 1.

*Tabel 3.3.2. Hasil Pengujian Ackerman vs Yeager*

| Percobaan | Perolehan Poin |        |
|-----------|----------------|--------|
|           | Ackerman       | Yeager |
| 1         | 1019           | 249    |
| 2         | 1384           | 516    |
| 3         | 916            | 318    |

|           |      |     |
|-----------|------|-----|
| 4         | 1227 | 282 |
| 5         | 794  | 287 |
| Rata-rata | 1068 | 274 |

Berikut adalah hasil pengujian yang dilakukan terhadap algoritma solusi Ackerman dengan algoritma solusi Reiner dengan tipe permainan 1 vs 1.

*Tabel 3.3.3 Hasil Pengujian Ackerman vs Reiner*

| Percobaan | Perolehan Poin |        |
|-----------|----------------|--------|
|           | Ackerman       | Reiner |
| 1         | 454            | 331    |
| 2         | 374            | 296    |
| 3         | 570            | 288    |
| 4         | 333            | 292    |

|           |       |       |
|-----------|-------|-------|
| 5         | 451   | 264   |
| Rata-rata | 436,4 | 294,2 |

Berikut adalah hasil pengujian yang dilakukan terhadap algoritma solusi Ackerman dengan algoritma solusi Reiss dengan tipe permainan 1 vs 1.

*Tabel 3.3.4 Hasil Pengujian Ackerman vs Reiss*

| Percobaan | Perolehan Poin |       |
|-----------|----------------|-------|
|           | Ackerman       | Reiss |
| 1         | 374            | 215   |
| 2         | 537            | 85    |
| 3         | 505            | 36    |
| 4         | 276            | 60    |
| 5         | 251            | 149   |

|           |       |     |
|-----------|-------|-----|
| Rata-rata | 388,6 | 109 |
|-----------|-------|-----|

### 3.4. Strategi Greedy Terpilih

Berdasarkan hasil analisis, algoritma Ackerman dipilih sebagai algoritma solusi yang terbaik. Keputusan ini didasarkan pada beberapa faktor berikut:

#### 1. Kinerja Terbaik dalam Pengujian

Dari hasil pengujian yang dilakukan, algoritma Ackerman menunjukkan perolehan poin rata-rata tertinggi dibandingkan dengan algoritma lainnya. Dalam uji coba 1 vs 1 dengan bot Yeager, Reiner, dan Reiss, Ackerman secara konsisten meraih hasil yang lebih baik, dengan perolehan poin yang signifikan lebih tinggi.

#### 2. Kemampuan Adaptasi terhadap Situasi

Algoritma Ackerman berhasil beradaptasi dengan baik terhadap berbagai situasi di arena. Kemampuan bot untuk mengatur posisi dan mengarahkan tembakan berdasarkan posisi musuh terbukti efektif, yang memberikan keuntungan dalam jangka panjang selama pertarungan. Ini menunjukkan karakteristik bot Ackerman yang lebih fleksibel dalam menanggapi perubahan dinamika dalam permainan.

#### 3. Efisiensi dalam Penggunaan Energi

Meskipun fokus utama algoritma ini adalah peningkatan frekuensi penembakan, algoritma Ackerman juga mengoptimalkan penggunaan energi dengan menyesuaikan frekuensi tembakan berdasarkan posisi musuh. Hal ini memberikan keuntungan dalam mempertahankan energi bot selama pertandingan yang lebih lama. Pengaturan tembakan ini



menunjukkan bahwa Ackerman memiliki keseimbangan yang baik antara agresivitas dan efisiensi energi, yang tidak dimiliki oleh bot lainnya.

#### **4. Keunggulan dalam Kompetisi**

Melalui uji coba dan perbandingan terhadap algoritma lainnya, Ackerman menunjukkan keunggulan dalam hal total perolehan poin rata-rata. Pengujian 1 vs 1 dengan bot lain menunjukkan Ackerman dapat mengatasi tantangan yang diberikan oleh bot lainnya dengan strategi yang lebih unggul. Keunggulan ini tidak hanya berdasarkan perolehan poin, tetapi juga dalam cara Ackerman mengelola pergerakan dan serangan selama pertempuran.

Namun, kami juga melihat dari ciri dan karakteristik unik Bot Ackerman yang bisa menjadi alasan ditetapkan sebagai bot utama yang baik dan efektif.

##### **1. Strategi Tembakan dan Posisi**

Salah satu karakteristik utama dari bot Ackerman adalah kemampuannya untuk melakukan penembakan berkelanjutan sambil memanfaatkan posisi musuh yang lebih mudah dijangkau dengan teknik *tracking*. Dengan menggunakan strategi *tracking* dan pencarian posisi yang aman, Ackerman dapat mengendalikan pertarungan lebih efektif.

##### **2. Mampu beradaptasi dengan ciri dan karakteristik musuh secara menyeluruh**

Ackerman memiliki keunggulan dalam hal kecepatan adaptasi terhadap posisi musuh dan kondisi arena. Pengaturannya terhadap tembakan dan pergerakan berdasarkan perubahan jarak musuh menunjukkan strategi yang lebih dinamis dan responsif dibandingkan dengan bot lainnya. Misalnya, Ackerman mengurangi atau meningkatkan

tembakan berdasarkan kedekatannya dengan musuh, serta dapat menghindari serangan ketika mendapat serangan.

### **3. Energi yang terbuang banyak justru dapat memberikan poin banyak dan penambahan poin**

Dalam hal pengelolaan energi, Ackerman mungkin relatif melakukan pemborosan tembakan, namun bot ini menyesuaikan tingkat tembakan dengan jarak dan status energi, sehingga tidak menghabiskan energi terlalu cepat atau memberikan kesempatan bagi musuh untuk menyerang dengan mudah. Apabila berada di jarak dekat, maka kemungkinan perolehan poin nya tinggi dan energinya akan justru bertambah.

Berdasarkan alasan-alasan tersebut, algoritma Ackerman dipilih sebagai solusi terbaik untuk mencapai tujuan dalam permainan. Keunggulan-keunggulan yang telah disebutkan di atas, baik dari segi uji permainan dalam pertempuran dan ciri karakteristik, menjadikannya pilihan yang lebih unggul dibandingkan dengan algoritma lainnya.

## Bab 4: Implementasi dan Pengujian

### 4.1 Implementasi

#### 4.1.1. Ackerman (Bot Utama)

Bot ini menerapkan algoritma greedy untuk menyerang musuh dengan fokus utama pada penembakan peluru yang lebih banyak serta memanfaatkan posisi musuh yang lebih mudah terkena tembakan, dengan tujuan agar energi musuh cepat habis dan energi bot bertambah. Bot ini dominan menuju area yang lebih sepi, sambil melakukan serangan *spray* dari jarak jauh, tetapi tetap bergerak untuk menghindari tembakan musuh. Ketika sudah bergerak karena terkena peluru musuh, bot akan lebih memilih corner secara acak sebagai tujuan pergerakan. Saat menyerang musuh, bot akan menembakkan peluru berdasarkan jarak musuh yang terdeteksi, baik dengan diam atau bergerak mendekati musuh. Jika tidak ada musuh yang terdeteksi, bot akan memperluas area pemindaian dalam rentang tertentu. Selain itu, jika bot terkena tembakan atau menabrak dinding, bot akan bergerak menjauh dan mencari arah yang berlawanan untuk menghindari bahaya lebih lanjut. Implementasinya dalam pseudocodenya adalah sebagai berikut :

```
// Inisialisasi warna bot dan senjata  
  
Set BodyColor = Dark Raven Black (RGB: 42, 42, 42)  
  
Set TurretColor = Dark Raven Blue (RGB: 16, 32, 48)  
  
Set RadarColor = Blue  
  
Set BulletColor = Midnight (RGB: 0, 0, 102)  
  
Set ScanColor = Navy (RGB: 0, 0, 128)
```

```

// Loop utama bot
While IsRunning

    Set enemies = EnemyCount

    Set gunIncrement = 3

    Set noEnemyCounter = 0

    // Putar senjata 360 derajat mencari musuh

    For i = 0 to 59

        TurnGunRight(gunIncrement) // Putar senjata

        If Musuh Terdeteksi

            Set noEnemyCounter = 0

        Else

            Increment noEnemyCounter

    // Jika tidak ada musuh terdeteksi setelah beberapa waktu,
    arah senjata

    If noEnemyCounter >= 5 * 30

        gunIncrement = Min(gunIncrement + 30, 180)

        Set noEnemyCounter = 0

        MoveToLessCrowdedArea()

```

```

        // Balikkan arah putaran senjata

        gunIncrement *= -1
End While

// Fungsi untuk menangani musuh yang terdeteksi
OnScannedBot(e)

    // Hitung sudut senjata relatif terhadap musuh

    bearingFromGun = GunBearingTo(e.X, e.Y)

    // Putar senjata untuk menghadap musuh

    TurnGunLeft(bearingFromGun)

    // Jika sudah menghadap musuh dan senjata siap, tembakkan

    If (abs(bearingFromGun) <= 3 and GunHeat == 0)

        Fire(Min(3 - abs(bearingFromGun), Energy - .1))

    // Jika bot sudah menghadap langsung ke musuh, lakukan
    aian ulang

    If bearingFromGun == 0

        Rescan()

// Fungsi untuk menangani bot yang terkena peluru

```

```

OnHitByBullet(evt)

    // Hitung arah peluru yang mengenai bot

    bearing = CalcBearing(evt.Bullet.Direction)

    // Putar bot untuk menghadap arah peluru dan mundur

    TurnLeft(90 - bearing)

    Back(5000)

// Fungsi untuk bergerak ke area yang kurang ramai
MoveToLessCrowdedArea()

    // Dapatkan musuh terdekat

    closestBot = GetClosestBot()

    // Jika ada musuh terdekat, bergerak ke arah berlawanan

    If closestBot != null

        angleToEnemy = CalcBearingTo(closestBot.X, closestBot.Y)

        TurnLeft(angleToEnemy + 180)

        SetForward(100)

        Forward(100)

    Else

        MoveToCorner()

```

```

// Fungsi untuk bergerak ke sudut arena

MoveToCorner()

    // Hitung arah menuju sudut arena yang dipilih secara acak

    angleToCorner = CalcBearingToCorner(corner)

    TurnLeft(angleToCorner)

    SetForward(1000)

    Forward(1000)

// Fungsi untuk menghitung sudut antara bot dan musuh

CalcBearingTo(x, y)

    dx = x - X

    dy = y - Y

    return Math.Atan2(dy, dx) * (180 / Math.PI) - Direction

// Fungsi untuk menghitung sudut ke sudut arena

CalcBearingToCorner(corner)

    switch corner

        case 0: cornerX = 0, cornerY = 0

        case 90: cornerX = ArenaWidth, cornerY = 0

        case 180: cornerX = ArenaWidth, cornerY = ArenaHeight

        case 270: cornerX = 0, cornerY = ArenaHeight

```

```

        return CalcBearingTo(cornerX, cornerY)

// Fungsi untuk mendapatkan bot terdekat
GetClosestBot()

    closestBot = null

    closestDistance = double.MaxValue

    // Iterasi melalui setiap musuh yang terdeteksi
    For each bot in Events.OfType<ScannedBotEvent>()

        distance = Math.Sqrt(Math.Pow(bot.X - X, 2) +
        Math.Pow(bot.Y - Y, 2))

        If distance < closestDistance

            closestDistance = distance

            closestBot = bot

    return closestBot

// Fungsi untuk memilih sudut secara acak ke sudut arena
RandomCorner()

    Random rand = new Random()

    return rand.Next(4) * 90 // Pilih secara acak antara 0, 90,
    180, dan 270 derajat

```



```
// Fungsi untuk menangani tabrakan dengan dinding

OnHitWall(e)

    ReverseDirection() // Balikkan arah jika menabrak dinding


// Fungsi untuk membalikkan arah bot

ReverseDirection()

    If movingForward

        SetBack(40000) // Mundur jika sedang bergerak maju

        movingForward = false

    Else

        SetForward(40000) // Maju jika sedang mundur

        movingForward = true
```

#### 4.1.2. Reiss

Bot ini menggunakan algoritma Greedy dalam strategi penyerangannya, dengan fokus utama pada efisiensi penggunaan energi. Bot tetap akan menyerang musuh, namun tingkat kerusakan yang diberikan akan bervariasi tergantung pada jarak musuh yang terdeteksi. Selain itu, jumlah peluru yang ditembakkan juga disesuaikan dengan kondisi sekitar. Jika jumlah musuh yang terdeteksi sedikit, bot akan menggunakan sistem satu peluru per tembakan untuk menghemat energi. Sebaliknya, jika musuh dalam jumlah banyak, bot akan menembakkan lebih dari satu peluru untuk meningkatkan efektivitas serangan. Selain mekanisme menyerang, bot ini juga memiliki strategi bertahan. Jika terkena tembakan, bot akan segera bergerak ke arah yang berlawanan untuk menghindari serangan

lanjutan dan meningkatkan peluang bertahan lebih lama di arena. Implementasi dalam pseudocodenya adalah sebagai berikut :

```
// Inisialisasi warna bot dan senjata

Set BodyColor = Oregano Green (RGB: 108, 154, 92)

Set TurretColor = Rain Blue (RGB: 0, 82, 138)

Set RadarColor = Cyan

Set BulletColor = Azul Marino (RGB: 0, 56, 100)

Set ScanColor = Magenta


// Loop untuk pergerakan bot

While IsRunning

    // Maju sejauh 100 units

    Forward(100)

    // Putar senjata 360 derajat untuk mencari musuh

    TurnGunRight(360)

    // Mundur sejauh 100 units

    Back(100)

    // Putar senjata 360 derajat lagi

    TurnGunRight(360)


// Fungsi untuk menangani musuh yang terdeteksi
```

```

OnScannedBot (e)

    // Hitung sudut dari posisi musuh relatif terhadap senjata bot

    bearingFromGun = GunBearingTo(e.X, e.Y)

    distance = HitungJarak(e.X, e.Y)


    // Putar senjata sesuai sudut yang dihitung

    TurnGunLeft(bearingFromGun)


    // Jika bot sudah menghadap musuh dan senjata siap, tembakkan

    Jika (abs(bearingFromGun) <= 3 dan GunHeat == 0)

        Fire(Min(3 - abs(bearingFromGun), Energy - .1))


    // Jika senjata sudah menghadap langsung ke musuh, lakukan
    lang

    Jika bearingFromGun == 0

        Rescan()


    // Panggil SmartFire berdasarkan jarak dan energi musuh

    SmartFire(distance, e.Energy)


// Fungsi untuk menangani ketika bot terkena peluru

OnHitByBullet(evt)

```

```

// Hitung arah peluru yang mengenai bot

bearing = CalcBearing(evt.Bullet.Direction)


// Putar bot untuk menghadap arah peluru dan mundur

TurnLeft(270 - bearing)

Back(3000)


// Fungsi untuk menentukan kekuatan tembakan berdasarkan jarak dan
musuh
SmartFire(distance, enemyEnergy)

// Tentukan kekuatan tembakan berdasarkan jarak

Jika distance > 200

    firePower = 1

Jika distance > 50

    firePower = 2

Jika distance <= 50

    firePower = 3


// Sesuaikan kekuatan tembakan jika energi bot rendah

Jika Energy < 15

    firePower = Min(firePower, 1)

```

```

// Sesuaikan kekuatan tembakan jika energi musuh rendah

Jika enemyEnergy < 10

    firePower = Min(firePower, 1)

// Tembak dengan kekuatan yang dihitung

Fire(firePower)

```

#### 4.1.3. Reiner

Bot ini menerapkan algoritma greedy dalam menyerang. Dalam menyerang, bot akan mencari musuh yang memiliki energi terendah dan menjadikannya sebagai target. Hal ini karena akan lebih mudah untuk mengalahkan bot dengan energi terendah dan akan mengurangi jumlah bot musuh yang ada. Jika terdapat bot dengan jumlah energi terendah yang sama bot akan menghitung jarak menuju bot musuh tersebut maka bot musuh dengan jarak akan terpilih menjadi target. Dalam pergerakan, bot akan bergerak maju dan mundur dan memindai area permainan, jika bot mendeteksi musuh bot akan menyesuaikan arah tembakan dan menyerang sesuai strategi greedy. Jika terdapat bot musuh yang menabrak bot, ia akan menyesuaikan arah tembakan dan melakukan tembakan. Selain itu, jika bot tertembak musuh atau menabrak dinding, bot akan membalik arah dengan melakukan rotasi untuk menghindari tembakan berulang dari musuh. Implementasi bot dalam pseudocode sebagai berikut:

```

FUNCTION Main()

    INITIALIZE bot as Reiner

```

```

START bot

CLASS Reiner EXTENDS Bot

    VARIABLES:

        movingForward as boolean

        Dictionary<int, (double x, double y, double energy)>
        enemyBots

CONSTRUCTOR Reiner()

    CALL base(BotInfo.FromFile("Reiner.json"))

FUNCTION Run()

    SET warna bot (Body: Red, Turret: White, Radar: White,
scan: Yellow)

    movingForward <- TRUE

    WHILE IsRunning
        TURN Gun RIGHT 360

        IF movingForward = TRUE THEN

            MOVE Forward 100

        ELSE

            MOVE Back 100

    EVENT OnScannedBot(e)

        enemyBots[e.ScannedBotId] <- (e.X, e.Y, e.Energy)

```

```

        scannedDistance <- DistanceTo(e.X, e.Y)

        bearingFromGun <- GunBearingTo(e.X, e.Y)

        TURN Gun Left (bearingFromGun)

    IF scannedDistance < 100 THEN

        bearingFromGun <- GunBearingTo(e.X, e.Y)

        TURN Gun Left(bearingFromGun)

        FIRE 2

    IF e.Energy <= 0 THEN

        enemyBots.Remove(e.scannedBotId)

    ELSE

        enemyBots[e.scannedBotId] <- (e.X, e.Y, e.Energy)

    CALL AttackLowestEnergy()

FUNCTION AttackLowestEnergy()

    Target <- GetLowestEnergy()
    bearingFromGun <- GunBearingTo(Target.X, Target.Y)

    TURN Gun Left(bearingFromGun)
    enemyDistance <- DistanceTo(Target.X, Target.Y)

    IF enemyDistance < 75 THEN

        Fire 3

```

```

ELSE IF enemyDistance < 150
    Fire 2

ELSE

    FIRE 1

FUNCTION GetLowestEnergy()

    minEnergy <- MaxValue
    minDistance <- MaxValue

    TargetX <- 0

    TargetX <- 0

    targetEnergy <- 0

    FOR EACH enemy in enemyBots.value
        enemyDistance <- DistanceTo(enemy.X, enemy.Y)

        IF enemy.energy < minEnergy OR (enemy.energy ==
minEnergy AND enemyDistance < minDistance)

            TargetX <- enemy.X
            TargetY <- enemy.Y

            targetEnergy <- enemy.energy

            minEnergy <- enemy.energy

            minDistance <- enemyDistance

    RETURN (targetX, TargetY, targetEnergy)

EVENT OnHitByBullet(e)

```



```

CALL ReverseDirection()

TURN Right 90
MOVE Forward 100

TURN Left 90

EVENT OnHitBot(e)

    direction <- DirectionTo(e.X, e.Y)
    gunBearing <- NormalizeRelativeAngle(direction +
GunDirection)
    TURN Gun Left (gunBearing)
    Fire 2
    MOVE Forward 200

EVENT OnHitWall(e)

    CALL ReverseDirection()

FUNCTION ReverseDirection()

    IF movingForward = TRUE THEN

        MOVE Back 400

        movingForward <- FALSE

    ELSE

        MOVE Forward 400

        movingForward <- TRUE

```

#### 4.1.4. Yeager

Bot Yeager menerapkan strategi greedy berbasis energi yang berarti pengambilan keputusan bot ini didasarkan pada *energy level* dari sang bot itu sendiri. Implementasi ini digunakan dengan cara membagikan tindakan bot menjadi 3 fase yakni *AttackPhase* (fase menyerang), *DefensivePhase* (fase defensif), dan *NothingToLosePhase* (fase tanpa ada pertimbangan lebih lanjut). Masing-masing fase memiliki metodenya sendiri, dari segi pertimbangan energi sendiri bot akan melaksanakan *AttackPhase* ketika *energy level* berada pada angka di atas sama dengan 60, *DefensivePhase* dilaksanakan ketika *energy level* berada pada angka di bawah 60 dan di atas sama dengan 25, lalu apabila *energy level* berada di bawah 25 maka bot akan memasuki fase *NothingToLosePhase*. Bot Yeager memiliki perilaku yang adaptif dan fleksibel, yang disesuaikan dengan kondisi energi dan situasi di arena. Berikut adalah perilaku utama bot Yeager:

##### 1. Pergerakan

- *AttackPhase*: Bot bergerak maju untuk mendekati musuh sambil memindai area sekitar.
- *DefensivePhase*: Bot bergerak menjauh dari musuh dan menuju sudut arena yang dipilih secara acak sebagai zona aman.
- *NothingToLosePhase*: Bot bergerak secara acak tanpa tujuan yang jelas.

##### 2. Penembakan

- Bot akan menembak musuh yang terdeteksi oleh radar. Kekuatan tembakan disesuaikan dengan jarak dan energi bot.

- Jika musuh berada dalam jangkauan dan senjata tidak terlalu panas, bot akan menembak dengan kekuatan yang optimal.

### 3. Penghindaran

- Jika bot menabrak dinding, bot akan membalik arah dan memutar senjata untuk menghindari tabrakan lebih lanjut.
- Jika bot terkena tembakan musuh, bot akan bergerak menjauh dari arah serangan untuk menghindari kerusakan lebih lanjut.

Berikut adalah tampilan kodenya dalam bentuk pseudocode:

```
FUNCTION Main()  
  
    INITIALIZE bot as Yeager  
  
    START bot  
  
CLASS Yeager EXTENDS Bot  
  
    VARIABLES:  
  
        movingForward as boolean  
  
        currentEnemyX, currentEnemyY as double  
  
        corner as integer  
  
    CONSTRUCTOR Yeager()  
  
        CALL base(BotInfo.FromFile("Yeager.json"))  
  
        corner <- RandomCorner() // Pilih sudut awal secara acak
```

```

FUNCTION Run()

    SET warna bot (Body: Blue, Turret: Red, Radar: Yellow,
Bullet: Green)

    movingForward <- TRUE

    WHILE IsRunning

        CALL CheckEnergyAndSwitchPhase()

        CALL MoveToSafeZone()

        TURN Gun Right 360 (Scanning)

FUNCTION CheckEnergyAndSwitchPhase()

    IF Energy >= 60 THEN

        CALL AttackPhase()

    ELSE IF Energy < 60 AND Energy >= 25 THEN

        CALL DefensivePhase()

    ELSE IF Energy < 25 AND Energy > 0 THEN

        CALL NothingToLosePhase()

FUNCTION AttackPhase()

    MOVE Forward 100

    TURN Gun Right 360

```

```

FUNCTION DefensivePhase()

    TURN Gun Right 360

    IF currentEnemyX  $\neq$  0 AND currentEnemyY  $\neq$  0 THEN

        bearingFromBot <- BearingTo(currentEnemyX,
currentEnemyY)

        moveDirection <-
NormalizeRelativeAngle(bearingFromBot + 180)

        TURN Left (moveDirection - Direction)

        MOVE Forward 100

FUNCTION NothingToLosePhase()

    TURN Gun Right 360

    FIRE 1

EVENT OnScannedBot(e)

    currentEnemyX <- e.X

    currentEnemyY <- e.Y

    bearingFromGun <- GunBearingTo(e.X, e.Y)

    TURN Gun Left (bearingFromGun)

    IF |bearingFromGun|  $\leq$  3 AND GunHeat = 0 THEN

```

```

        FIRE MIN(3 - |bearingFromGun|, Energy - 0.1)

    IF bearingFromGun = 0 THEN

        CALL Rescan()

EVENT OnHitWall(e)

    CALL ReverseDirection()

    TURN Right 180

    TURN Left 90

    TURN Gun Right 360

FUNCTION ReverseDirection()

    IF movingForward = TRUE THEN

        MOVE Back 40000

        movingForward <- FALSE

    ELSE

        MOVE Forward 40000

        movingForward <- TRUE

FUNCTION MoveToSafeZone()

    IF Energy < 40 THEN

        safeX <- ArenaWidth / 4 + (corner % 2) * ArenaWidth /

```

```

2
    safeY <- ArenaHeight / 4 + (corner / 2) * ArenaHeight
/ 2

    bearingToSafeZone <- BearingTo(safeX, safeY)

    moveDirection <-
NormalizeRelativeAngle(bearingToSafeZone - Direction)

    TURN Left moveDirection

    MOVE Forward 100

FUNCTION RandomCorner()

    RETURN Random(0,3)

```

## 4.2 Penjelasan Struktur Data, Fungsi, dan Prosedur

### 4.2.1 Bot Ackerman

#### 4.2.1.1. Atribut Kelas Ackerman

| Nama Atribut | Deskripsi |
|--------------|-----------|
|--------------|-----------|

|                                                                         |                                                                                                                                                                                                                             |
|-------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>int enemies</code>                                                | Menyimpan jumlah musuh yang terdeteksi oleh radar bot. Nilai ini diperoleh dari properti <code>EnemyCount</code> , yang dihitung secara otomatis oleh sistem Robocode untuk jumlah bot yang ada di arena.                   |
| <code>int corner</code>                                                 | Menyimpan nilai yang menunjukkan sudut corner acak yang dipilih oleh bot. Corner ini digunakan untuk menentukan arah bot bergerak pada awal permainan.                                                                      |
| <code>bool movingForward</code>                                         | Menyimpan status apakah bot sedang bergerak maju atau mundur. Variabel ini digunakan pada saat bot menabrak dinding untuk menentukan apakah bot harus bergerak maju atau mundur agar bisa menghindari tabrakan selanjutnya. |
| <code>BodyColor, TurretColor, RadarColor, BulletColor, ScanColor</code> | Atribut-atribut ini digunakan untuk menentukan warna dari berbagai bagian bot. Ini memberi pengaruh                                                                                                                         |



|  |                                    |
|--|------------------------------------|
|  | pada tampilan visual bot di arena. |
|--|------------------------------------|

#### 4.2.1.1. Fungsi dan Prosedur Utama Ackerman

| Nama Atribut | Deskripsi                                                                                                                                                                                                                                 |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Main ()      | Fungsi ini merupakan titik masuk dari program. Ini akan membuat objek bot Ackerman dan memulai eksekusi bot dengan memanggil metode Start(). Program ini dimulai dengan eksekusi bot yang sudah dikonfigurasi.                            |
| Run ()       | <p>Fungsi ini adalah metode utama yang menjalankan logika dari bot Ackerman. Dalam Run():</p> <ul style="list-style-type: none"> <li>- Set warna bot dan elemen-elemen lainnya seperti tubuh, turret, radar, peluru, dan scan.</li> </ul> |

|                 |                                                                                                                                                                                                                                                                                                                                                                                                            |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                 | <ul style="list-style-type: none"> <li>- Bot akan memindahkan turret untuk mencari musuh dan menembak jika musuh terdeteksi.</li> <li>- Menggunakan prinsip greedy untuk mencari area yang lebih sepi setelah beberapa waktu tidak mendeteksi musuh dan berusaha bergerak menuju area tersebut.</li> </ul>                                                                                                 |
| OnScannedBot () | <p>Metode ini dipanggil ketika bot mendeteksi musuh. Di dalamnya:</p> <ul style="list-style-type: none"> <li>- Menghitung arah tembak (bearing) dan menyesuaikan arah turret.</li> <li>- Menembak musuh jika berada dalam jarak yang dapat dijangkau dengan peluru (dengan mempertimbangkan GunHeat dan energi).</li> <li>- Jika musuh berada tepat di depan bot, bot akan memindai ulang untuk</li> </ul> |

|                                                           |                                                                                                                                                                                                                                                                                                                                            |
|-----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                           | mencari musuh lain.                                                                                                                                                                                                                                                                                                                        |
| OnHitByBullet()                                           | <p>Fungsi ini menangani ketika bot terkena peluru dari musuh. Bot akan:</p> <ul style="list-style-type: none"> <li>- Menghitung arah tembakan musuh.</li> <li>- Memutar tubuh bot 90 derajat ke arah yang berlawanan dengan arah tembakan.</li> <li>- Bergerak mundur sejauh 5000 unit untuk menghindari serangan lebih lanjut.</li> </ul> |
| RandomCorner(),<br>OnHitWall(), dan<br>ReverseDirection() | Kedua fungsi ini digunakan untuk corner secara acak pada awal permainan dan membalik arah bot saat terjadi tabrakan dengan dinding.                                                                                                                                                                                                        |

#### 4.2.1.1. Fungsi dan Prosedur Alternatif Ackerman

| Nama Atribut                                                        | Deskripsi                                                                                                                                                                                                                                                                                    |
|---------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>MoveToLessCrowdedArea()</code>                                | <p>Fungsi ini menggerakkan bot ke area yang lebih sepi:</p> <ul style="list-style-type: none"> <li>- Jika ada bot terdekat, bot akan bergerak menjauh dari bot tersebut.</li> <li>- Jika tidak ada musuh, bot akan bergerak menuju corner yang telah dipilih pada awal permainan.</li> </ul> |
| <code>MoveToCorner()</code>                                         | Fungsi ini menggerakkan bot menuju corner yang telah dipilih sebelumnya pada awal permainan.                                                                                                                                                                                                 |
| <code>GetClosestTo()</code>                                         | Fungsi ini akan mendapatkan bot musuh terdekat dengan melakukan scanning                                                                                                                                                                                                                     |
| <code>CalcBearingTo()</code> dan <code>CalcBearingToCorner()</code> | Kedua fungsi ini digunakan untuk menghitung arah (bearing) bot                                                                                                                                                                                                                               |

|                                                                                                    |                                                                                                                                     |
|----------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                    | menuju titik tertentu, baik itu musuh atau corner. Fungsi ini penting untuk pergerakan dan penargetan bot.                          |
| <code>RandomCorner()</code> ,<br><code>OnHitWall()</code> , dan<br><code>ReverseDirection()</code> | Kedua fungsi ini digunakan untuk corner secara acak pada awal permainan dan membalik arah bot saat terjadi tabrakan dengan dinding. |

## 4.2.2 Bot Reiss

### 4.2.2.1. Atribut Kelas Reiss

| Nama Atribut                                                                                                                          | Deskripsi                                                                                                                              |
|---------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <code>BodyColor</code> , <code>TurretColor</code> ,<br><code>RadarColor</code> ,<br><code>BulletColor</code> , <code>ScanColor</code> | Atribut-atribut ini digunakan untuk menentukan warna dari berbagai bagian bot. Ini memberi pengaruh pada tampilan visual bot di arena. |

#### 4.2.2.2. Metode Utama Reis

| Nama Atribut                      | Deskripsi                                                                                                                                                                                                                                                                                                                                              |
|-----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Run ( )                           | Fungsi Run() adalah inti dari pergerakan bot yang secara berulang-ulang bergerak maju sejauh 100 unit, memutar senjatanya selama 360 derajat, bergerak mundur sejauh 100 unit, dan kemudian memutar senjata kembali. Bot akan melakukan pemindaian dan pergerakan secara terus-menerus dalam arena untuk mendeteksi musuh.                             |
| OnScannedBot (ScannedBot Event e) | OnScannedBot ( ) dipanggil ketika bot mendeteksi musuh. Bot akan menghitung bearing (arah relatif) dan jarak ke musuh, memutar senjata untuk menembak, dan mengevaluasi energi bot dan musuh. Jika musuh terdeteksi tepat di depan, bot akan memindai ulang (Rescan). Tembakan dilakukan dengan kekuatan yang disesuaikan berdasarkan jarak dan energi |

|                                                   |                                                                                                                                                                                                                                                                       |
|---------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                   | musuh.                                                                                                                                                                                                                                                                |
| <code>OnHitByBullet (HitByBulletEvent evt)</code> | Fungsi ini menangani kondisi ketika bot terkena peluru dari musuh. <code>CalcBearing</code> digunakan untuk menghitung arah dari peluru yang mengenai bot. Bot kemudian memutar dirinya untuk menghadap arah peluru dan mundur untuk menghindari tembakan berikutnya. |

#### 4.2.2.3. Metode Alternatif Reiss

| Nama Atribut                                                 | Deskripsi                                                                                                                                                                                                                                                                                          |
|--------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>SmartFire (double distance, double enemyEnergy)</code> | Fungsi <code>SmartFire</code> memilih kekuatan tembakan berdasarkan jarak ke musuh. Jika musuh jauh, tembakan lebih lemah; jika dekat, tembakan lebih kuat. Jika energi bot atau energi musuh rendah, kekuatan tembakan dibatasi untuk menjaga keselamatan bot atau menghindari pemborosan energi. |

### 4.2.3 Bot Reiner

#### 4.2.3.1. Atribut Kelas Reiner

Atribut bot adalah variabel atau properti yang digunakan untuk menyimpan informasi penting tentang bot selama permainan berlangsung. Berikut adalah atribut yang digunakan oleh bot Reiner :

| Nama Atribut                                                                              | Deskripsi                                                                                                                                                                                                   |
|-------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>private bool<br/>movingForward;</pre>                                                | Variabel boolean yang menyimpan status apakah bot sedang bergerak maju atau mundur. Ini digunakan untuk mengontrol arah pergerakan bot, terutama saat bot menabrak dinding.                                 |
| <pre>Private<br/>Dictionary&lt;int, (double<br/>x, double y, double<br/>energy)&gt;</pre> | Menyimpan informasi musuh yang terdeteksi oleh radar bot. Informasi yang disimpan berupa IDmusuh, posisi musuh dalam koordinat x dan y, dan energi musuh. Informasi ini diperoleh dari fungsi OnScannedBot. |



#### 4.2.3.2 Metode Utama Reiner

Fungsi utama bot adalah metode yang di-override dari kelas dasar Bot yang disediakan oleh Robocode Tank Royale API. Fungsi-fungsi ini adalah inti dari perilaku bot dan dipanggil secara otomatis oleh game engine.

| Metode                                        | Deskripsi                                                                                                                                                                                                                                                                                                                                                                                                      |
|-----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>Run ()</code>                           | <p>Fungsi ini adalah metode utama yang menjalankan logika dari bot Reiner. Dalam <code>Run ()</code> :</p> <ul style="list-style-type: none"><li>- Bot akan memutar turret untuk mencari musuh dan menembak jika musuh terdeteksi.</li><li>- Menggunakan prinsip greedy untuk mencari area yang lebih sepi setelah beberapa waktu tidak mendeteksi musuh dan berusaha bergerak menuju area tersebut.</li></ul> |
| <code>OnScannedBot (ScannedBotEvent e)</code> | <p>Metode ini dipanggil ketika bot mendeteksi musuh. Di dalamnya:</p> <ul style="list-style-type: none"><li>- Jika mendeteksi musuh, metode</li></ul>                                                                                                                                                                                                                                                          |

|                                     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                     | <p>akan menyimpan informasi posisi bot musuh dalam koordinat x, y dan energi yang dipunyai musuh.</p> <ul style="list-style-type: none"> <li>- Menghitung arah tembak (bearing) dan menyesuaikan arah turret.</li> <li>- Menembak musuh jika berada dalam jarak yang dapat dijangkau dengan peluru (dengan mempertimbangkan <i>gun heat</i> dan energi).</li> <li>- Jika musuh yang dideteksi memiliki energi <math>\leq 0</math> maka metode akan menghapus keberadaan bot dengan ID tersebut dalam dictionary. Namun, jika bot musuh masih mempunyai energi maka akan ditambahkan pada dictionary.</li> <li>- Di akhir metode akan memanggil prosedur <code>AttackLowestEnergy</code>.</li> </ul> |
| <code>AttackLowestEnergy ( )</code> | Prosedur ini menangani strategi menyerang bot musuh dengan energi                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |

|                                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                | <p>terendah. Pada prosedur ini bot akan:</p> <ul style="list-style-type: none"> <li>- Memanggil fungsi <code>GetLowestEnergy</code> dan menyimpan informasi dalam var <code>target</code>.</li> <li>- Menghitung arah tembak (bearing) dan menyesuaikan arah turret menuju koordinat musuh.</li> <li>- Mencari jarak bot musuh dengan <code>DistanceTo()</code> dan disimpan dalam <code>enemyDistance</code>.</li> <li>- Bot akan melakukan tembakan berdasarkan jarak musuh. Jika bot musuh berada dekat dengan bot maka kekuatan tembakan bot akan bertambah. Jika bot musuh berada jauh dari bot maka kekuatan tembakan berkurang.</li> </ul> |
| <code>GetLowestEnergy()</code> | <p>Fungsi ini akan memilih dan mencari musuh dengan energi terendah sebagai target. Pada fungsi ini bot akan:</p> <ul style="list-style-type: none"> <li>- Fungsi deklarasi terlebih dahulu, informasi yang akan dilakukan pengecekan seperti koordinat</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                |

|                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                            | <p>musuh dan energi yang dimiliki.</p> <ul style="list-style-type: none"> <li>- Melakukan pengecekan dengan iterasi. Nilai yang dilakukan pengecekan adalah energi yang dimiliki bot dan jarak bot musuh. Setelah mendapat bot yang memiliki energi terendah dan jarak terdekat, fungsi akan menyimpan informasi tersebut dan mengembalikan nilai yang diperlukan seperti, koordinat bot musuh dalam x, y dan energi yang dimiliki bot musuh.</li> </ul> |
| OnHitWall (HitWallEvent e) | <p>Fungsi ini menangani kondisi ketika bot menabrak tembok permainan. Fungsi ini akan memanggil fungsi lain yaitu fungsi reverseDirection(). Setelah itu bot akan memutar arah sejauh 90°.</p>                                                                                                                                                                                                                                                           |

#### 4.2.3.3 Metode Alternatif Reiner

Fungsi alternatif adalah metode tambahan yang dibuat oleh programmer untuk mendukung perilaku bot. Fungsi-fungsi ini tidak di-override dari API, tetapi digunakan untuk mengimplementasikan strategi bot Reiner.

| Metode                          | Deskripsi                                                                                                                                                                                    |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>ReverseDirection()</code> | Fungsi ini membalik arah bot saat menabrak dinding atau terkena serangan. Ini adalah strategi greedy karena bot mengambil keputusan untuk menghindari kerusakan dengan segera membalik arah. |

### 4.3.3 Bot Yeager

#### 4.3.3.1 Atribut Kelas Yeager

Atribut bot adalah variabel atau properti yang digunakan untuk menyimpan informasi penting tentang bot selama permainan berlangsung. Berikut adalah atribut yang digunakan oleh bot Yeager:

| Nama Atribut                                 | Deskripsi                                                                                                                                                          |
|----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>private bool<br/>movingForward;</code> | Variabel boolean yang menyimpan status apakah bot sedang bergerak maju atau mundur. Ini digunakan untuk mengontrol arah pergerakan bot, terutama saat bot menabrak |

|                                                                                 |                                                                                                                                                                                            |
|---------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                 | dinding.                                                                                                                                                                                   |
| <pre>private double currentEnemyX = 0;  private double currentEnemyY = 0;</pre> | Variabel ini menyimpan koordinat terakhir musuh yang terdeteksi oleh radar bot. Ini digunakan untuk menentukan arah serangan atau penghindaran.                                            |
| <pre>private int corner;</pre>                                                  | Variabel ini menyimpan sudut arena yang dipilih secara acak sebagai tujuan pergerakan bot saat berada dalam fase defensif. Ini adalah bagian dari strategi greedy untuk mencari zona aman. |

#### 4.3.3.2. Metode Utama Yeager

Fungsi utama bot adalah metode yang di-override dari kelas dasar Bot yang disediakan oleh Robocode Tank Royale API. Fungsi-fungsi ini adalah inti dari perilaku bot dan dipanggil secara otomatis oleh game engine.

| Metode | Deskripsi |
|--------|-----------|
|--------|-----------|

|                                   |                                                                                                                                                                                                                                                                                                                                                          |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Run ( )                           | Fungsi ini adalah loop utama bot yang akan terus berjalan selama bot masih aktif. Di dalamnya, bot akan memeriksa energi dan menentukan fase yang akan dijalankan (Attack, Defensive, atau NothingToLose). Ini adalah inti dari strategi greedy bot Yeager, karena bot mengambil keputusan berdasarkan kondisi energi saat ini.                          |
| OnScannedBot (ScannedBot Event e) | Fungsi ini dipanggil ketika bot mendeteksi musuh melalui radar. Bot akan menyimpan koordinat musuh, menghitung sudut tembak, dan menembak jika musuh berada dalam jangkauan. Ini adalah bagian dari strategi greedy karena bot mengambil keputusan untuk menyerang musuh yang terdeteksi tanpa mempertimbangkan musuh lain atau strategi jangka panjang. |
| OnHitWall (HitWallEvent e)        | Fungsi ini dipanggil ketika bot menabrak dinding. Bot akan membalik arah dan memutar senjata untuk menghindari                                                                                                                                                                                                                                           |

|  |                                                                                                                                                       |
|--|-------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | tabrakan lebih lanjut. Ini adalah bagian dari strategi greedy karena bot mengambil keputusan untuk menghindari kerusakan dengan segera membalik arah. |
|--|-------------------------------------------------------------------------------------------------------------------------------------------------------|

#### 4.3.4.1 Metode Alternatif Yeager

Fungsi alternatif adalah metode tambahan yang dibuat oleh programmer untuk mendukung perilaku bot. Fungsi-fungsi ini tidak di-override dari API, tetapi digunakan untuk mengimplementasikan strategi khusus bot Yeager.

| Metode                                   | Deskripsi                                                                                                                                                                                                                                                       |
|------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>CheckEnergyAndSwitchPhase()</code> | Fungsi ini memeriksa energi bot dan menentukan fase yang akan dijalankan ( <i>Attack</i> , <i>Defensive</i> , atau <i>NothingToLose</i> ). Ini adalah inti dari strategi greedy bot Yeager, karena bot mengambil keputusan berdasarkan kondisi energi saat ini. |
| <code>AttackPhase()</code>               | Fungsi ini dijalankan ketika energi bot tinggi ( $\geq 80$ ). Bot akan                                                                                                                                                                                          |



|                                   |                                                                                                                                                                                                                                                                                                               |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                   | bergerak maju dan memutar senjata untuk mencari musuh. Ini adalah strategi greedy karena bot mengambil keputusan untuk menyerang secara agresif tanpa mempertimbangkan konsekuensi jangka panjang.                                                                                                            |
| <code>DefensivePhase()</code>     | Fungsi ini dijalankan ketika energi bot sedang ( $25 \leq Energy < 75$ ). Bot akan bergerak menjauh dari musuh dan mencari zona aman. Ini adalah strategi greedy karena bot memilih untuk menghindari konflik langsung dan mencari posisi yang lebih aman untuk bertahan.                                     |
| <code>NothingToLosePhase()</code> | Fungsi ini dijalankan ketika energi bot sangat rendah ( $< 25$ ). Bot akan bergerak secara acak dan menembak musuh. Ini adalah strategi greedy karena bot mengambil keputusan untuk menyerang secara acak tanpa mempertimbangkan strategi jangka panjang, dengan harapan bisa mendapatkan poin sebelum energi |

|                                 |                                                                                                                                                                                                                  |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                 | habis.                                                                                                                                                                                                           |
| <code>MoveToSafeZone()</code>   | Fungsi ini menggerakkan bot ke zona aman (sudut arena) ketika energinya rendah. Ini adalah strategi greedy karena bot memilih untuk menghindari kerusakan dengan mencari posisi yang lebih aman.                 |
| <code>ReverseDirection()</code> | Fungsi ini membalik arah bot saat menabrak dinding atau terkena serangan. Ini adalah strategi greedy karena bot mengambil keputusan untuk menghindari kerusakan dengan segera membalik arah.                     |
| <code>RandomCorner()</code>     | Fungsi ini menghasilkan sudut acak untuk menentukan zona aman. Ini adalah bagian dari strategi greedy karena bot memilih sudut arena secara acak sebagai tujuan pergerakan saat tidak ada musuh yang terdeteksi. |

## 4.3. Pengujian

### 4.3.1. Hasil Pengujian

#### 4.3.1.1. Hasil Uji 1

| Results for 10 rounds |                  |             |          |             |             |              |          |           |      |      |      |
|-----------------------|------------------|-------------|----------|-------------|-------------|--------------|----------|-----------|------|------|------|
| Rank                  | Name             | Total Score | Survival | Surv. Bonus | Bullet Dmg. | Bullet Bonus | Ram Dmg. | Ram Bonus | 1sts | 2nds | 3rds |
| 1                     | Ackerman 1.0     | 816         | 300      | 60          | 380         | 75           | 0        | 0         | 6    | 0    | 0    |
| 2                     | Template Bot 1.0 | 0           | 0        | 0           | 0           | 0            | 0        | 0         | 0    | 6    | 0    |

#### 4.3.1.2. Hasil Uji 2

| Results for 10 rounds |                  |             |          |             |             |              |          |           |      |      |      |
|-----------------------|------------------|-------------|----------|-------------|-------------|--------------|----------|-----------|------|------|------|
| Rank                  | Name             | Total Score | Survival | Surv. Bonus | Bullet Dmg. | Bullet Bonus | Ram Dmg. | Ram Bonus | 1sts | 2nds | 3rds |
| 1                     | Ackerman 1.0     | 643         | 250      | 50          | 308         | 35           | 0        | 0         | 5    | 0    | 0    |
| 2                     | Template Bot 1.0 | 0           | 0        | 0           | 0           | 0            | 0        | 0         | 0    | 5    | 0    |

#### 4.3.1.3. Hasil Uji 3

| Results for 10 rounds |                  |             |          |             |             |              |          |           |      |      |      |
|-----------------------|------------------|-------------|----------|-------------|-------------|--------------|----------|-----------|------|------|------|
| Rank                  | Name             | Total Score | Survival | Surv. Bonus | Bullet Dmg. | Bullet Bonus | Ram Dmg. | Ram Bonus | 1sts | 2nds | 3rds |
| 1                     | Ackerman 1.0     | 478         | 200      | 40          | 206         | 31           | 0        | 0         | 4    | 0    | 0    |
| 2                     | Template Bot 1.0 | 0           | 0        | 0           | 0           | 0            | 0        | 0         | 0    | 4    | 0    |

### 4.3.2. Analisis Pengujian

Setelah dilakukan pengujian sebanyak 3 kali dengan bot utama (Ackerman) dengan bot asisten (TemplateBot) secara 1 vs 1. Apabila ditinjau dari *ranking* hasil akhir, maka bot utama selalu menunjukkan di posisi teratas dan bot asisten dibawahnya berturut turut. Hal ini

dikarenakan dari ciri karakteristik bot utama dan bot asisten secara terpisah. Ketika bot utama yang awalnya diam dapat melakukan scan terhadap bot asisten, maka bot utama dapat melakukan perlawanan terhadap bot asisten dengan tembakan *tracking*, dan bertindak defensif yaitu mundur dan bergeser ketika terkena peluru bot asisten, sesuai dengan ciri karakteristiknya. Namun, di sisi lain bot asisten juga tidak memiliki ciri karakteristik perlawanan dan perlindungan yang baik, karena hanya bergerak maju dan mundur beberapa langkah tanpa melakukan *scanning* keberadaan musuh, sehingga terasa mudah untuk mengalahkan bot asisten.

Meskipun begitu, penulis juga melihat beberapa hal yang optimal dan yang tidak optimal dari ciri karakteristik dari bot utama sendiri. Bot utama dapat melakukan tembakan *tracking* baik saat bergerak maupun diam sehingga arah tembakkannya dioptimalkan untuk mengenai bot asisten. Namun, pada teknik *scanning* yang kami implementasikan yaitu, awalnya berdiam dan melakukan *scanning* hanya 60 derajat dan bertambah seiring waktu ketika tidak terdeteksi musuh selama beberapa selang waktu hingga maksimal 180 derajat, yang awalnya agar diharapkan optimal dalam menentukan arah *tracking* tembakan selanjutnya setelah *scanning* sehingga tidak memerlukan waktu yang lama pun ternyata masih tidak sepenuhnya optimal, karena suatu waktu, bot utama tidak berhasil untuk *scanning* sama sekali apabila bot berada di jangkauan lebih dari 180 derajat dari *scanning* bot utama, sehingga harus mengenai terlebih dahulu bot utama agar bot utama bisa berpindah ke lain sisi dan melakukan *scanning* kembali. Ketidakefektifan ini juga mengarah kepada suatu kejadian unik, dimana tidak saling menyerang satu sama lain, namun hanya melihat energi bot mana dulu yang habis oleh karena tembakan sendiri, hal ini juga menjadi suatu faktor unik alasan bot utama menang dalam suatu *round* tertentu.

## **Bab 5: Kesimpulan dan Saran**

### **5.1. Kesimpulan**

Pembuatan empat bot yang menggunakan algoritma Greedy—Ackerman, Reiss, Reiner, dan Yeager—dalam permainan Robocode telah menunjukkan bahwa setiap bot memiliki kekuatan dan kelemahan masing-masing, tergantung pada strategi dan karakteristik masing-masing algoritma. Melalui pengujian yang dilakukan dalam berbagai skenario pertempuran, bot Ackerman terbukti memiliki performa yang paling unggul dengan perolehan poin rata-rata tertinggi dalam setiap percobaan. Keunggulan ini didapatkan berkat strategi penembakan yang agresif dan responsif terhadap posisi musuh yang dapat memaksimalkan potensi kerusakan. Sedangkan, bot Reiner mengoptimalkan strategi serangan dengan menargetkan musuh berenergi rendah dan menyesuaikan kekuatan tembakan berdasarkan jarak. Pendekatan ini meningkatkan efisiensi serangan, memungkinkan Reiner untuk memaksimalkan damage sambil mempertahankan akurasi dalam pertempuran.

Bot Reiss, Reiner, dan Yeager masing-masing memiliki pendekatan unik dalam menerapkan algoritma greedy, namun tidak termasuk dalam kategori bot utama karena keterbatasan strategi mereka dalam menghadapi situasi kompleks di arena. Bot Reiss unggul dalam efisiensi energi dan penyesuaian jumlah tembakan, tetapi kurang agresif dalam menargetkan musuh secara langsung. Bot Reiner berfokus pada eliminasi musuh dengan energi terendah, namun strategi ini bisa menjadi kelemahan jika musuh dengan energi lebih tinggi mendominasi pertempuran. Sementara itu, Bot Yeager memiliki mekanisme adaptif berbasis energi dengan tiga fase keputusan, tetapi kurang fleksibel dalam menghadapi perubahan strategi lawan yang tidak hanya bergantung pada energi. Ketiga bot ini menunjukkan efektivitas dalam kondisi tertentu, namun tidak memiliki keseimbangan optimal antara serangan, pertahanan, dan adaptasi, yang membuat mereka tidak dipilih sebagai bot utama dalam sistem pertempuran ini.

Secara keseluruhan, analisis yang dilakukan menunjukkan bahwa pemilihan strategi Greedy berbasis Ackerman adalah yang paling efektif dalam kondisi permainan yang bervariasi. Walaupun demikian, setiap bot menunjukkan potensi yang dapat dieksplorasi lebih lanjut dengan penyesuaian terhadap parameter-parameter tertentu dalam strategi mereka.

## **5.2. Saran**

Berdasarkan hasil implementasi dan pengujian bot-bot tersebut, beberapa saran yang dapat dipertimbangkan untuk pengembangan bot di masa depan adalah sebagai berikut:

### **1. Peningkatan Algoritma**

- Bot Reiss yang mengutamakan efisiensi energi dapat ditingkatkan dengan menambahkan strategi berbasis prediksi gerakan musuh untuk meningkatkan ketepatan dalam serangan.
- Bot Reiner dan Yeager memerlukan pengoptimalan lebih.

### **2. Pengujian Lebih Lanjut:**

- Disarankan untuk melakukan pengujian lebih banyak dengan variasi kondisi medan pertempuran yang lebih kompleks, serta bot-bot yang memiliki strategi lebih beragam.
- Menggunakan variasi dalam parameter waktu dan jarak tembak dapat membantu memahami lebih baik bagaimana bot merespon perubahan dalam dinamika permainan.

### **3. Penyempurnaan dalam Responsivitas Radar dan Senjata:**

- Agar bot dapat lebih efektif dalam mengunci dan mengikuti musuh, penting untuk mengembangkan sistem penyesuaian otomatis pada arah

radar dan senjata yang lebih fleksibel. Ini akan memungkinkan bot beradaptasi dengan pergerakan musuh dengan lebih cepat.

4. Implementasi Pembelajaran Mesin (Machine Learning):

- Untuk meningkatkan keefektifan setiap bot, bisa diterapkan algoritma machine learning untuk mengoptimalkan strategi pergerakan dan penembakan berdasarkan data dari hasil permainan sebelumnya.
- Pendekatan ini memungkinkan bot untuk mempelajari dan beradaptasi dengan pola-pola permainan musuh, yang akan meningkatkan kemampuan mereka seiring waktu.

## Lampiran

- Untuk GitHub repository tugas besar ini dapat diakses pada link [berikut](#).
- Link Video Youtube : [https://youtu.be/B\\_nLbizb\\_gY?si=9mKA4E-lkQIu7ThJ](https://youtu.be/B_nLbizb_gY?si=9mKA4E-lkQIu7ThJ)

| No | Poin                                                              | Ya                                  | Tidak |
|----|-------------------------------------------------------------------|-------------------------------------|-------|
| 1  | Bot dapat dijalankan pada Engine yang sudah dimodifikasi asisten. | <input checked="" type="checkbox"/> |       |
| 2  | Membuat 4 solusi greedy dengan heuristic yang berbeda.            | <input checked="" type="checkbox"/> |       |
| 3  | Membuat laporan sesuai dengan spesifikasi.                        | <input checked="" type="checkbox"/> |       |
| 4  | Membuat video bonus dan diunggah pada Youtube.                    | <input checked="" type="checkbox"/> |       |

| Results for 10 rounds |              |             |          |             |             |              |          |           |      |      |      |
|-----------------------|--------------|-------------|----------|-------------|-------------|--------------|----------|-----------|------|------|------|
| Rank                  | Name         | Total Score | Survival | Surv. Bonus | Bullet Dmg. | Bullet Bonus | Ram Dmg. | Ram Bonus | 1sts | 2nds | 3rds |
| 1                     | Reiss. 1.0   | 878         | 400      | 60          | 388         | 29           | 0        | 0         | 2    | 1    | 1    |
| 2                     | Ackerman 1.0 | 754         | 300      | 30          | 373         | 44           | 7        | 0         | 1    | 1    | 2    |
| 3                     | Reiner 1.0   | 680         | 450      | 0           | 204         | 4            | 22       | 0         | 0    | 3    | 2    |
| 4                     | Yegaer 1.0   | 652         | 200      | 30          | 386         | 32           | 4        | 0         | 2    | 0    | 0    |

| Results for 10 rounds |              |             |          |             |             |              |          |           |      |      |      |
|-----------------------|--------------|-------------|----------|-------------|-------------|--------------|----------|-----------|------|------|------|
| Rank                  | Name         | Total Score | Survival | Surv. Bonus | Bullet Dmg. | Bullet Bonus | Ram Dmg. | Ram Bonus | 1sts | 2nds | 3rds |
| 1                     | Ackerman 1.0 | 1119        | 400      | 60          | 583         | 60           | 16       | 0         | 2    | 1    | 1    |
| 2                     | Yegaer 1.0   | 986         | 400      | 30          | 482         | 70           | 4        | 0         | 2    | 2    | 0    |
| 3                     | Reiner 1.0   | 841         | 450      | 30          | 282         | 30           | 49       | 0         | 1    | 1    | 3    |
| 4                     | Reiss. 1.0   | 590         | 200      | 0           | 353         | 28           | 8        | 0         | 0    | 1    | 1    |



| Results for 10 rounds |              |             |          |             |             |              |          |           |      |      |      |
|-----------------------|--------------|-------------|----------|-------------|-------------|--------------|----------|-----------|------|------|------|
| Rank                  | Name         | Total Score | Survival | Surv. Bonus | Bullet Dmg. | Bullet Bonus | Ram Dmg. | Ram Bonus | 1sts | 2nds | 3rds |
| 1                     | Ackerman 1.0 | 888         | 350      | 30          | 451         | 50           | 6        | 0         | 1    | 2    | 1    |
| 2                     | Yegaer 1.0   | 847         | 350      | 60          | 380         | 51           | 5        | 0         | 2    | 1    | 0    |
| 3                     | Reiss. 1.0   | 628         | 250      | 30          | 322         | 19           | 6        | 0         | 2    | 0    | 1    |
| 4                     | Reiner 1.0   | 602         | 400      | 0           | 174         | 6            | 22       | 0         | 0    | 2    | 3    |

| Results for 10 rounds |              |             |          |             |             |              |          |           |      |      |      |
|-----------------------|--------------|-------------|----------|-------------|-------------|--------------|----------|-----------|------|------|------|
| Rank                  | Name         | Total Score | Survival | Surv. Bonus | Bullet Dmg. | Bullet Bonus | Ram Dmg. | Ram Bonus | 1sts | 2nds | 3rds |
| 1                     | Ackerman 1.0 | 629         | 250      | 30          | 305         | 40           | 5        | 0         | 1    | 1    | 0    |
| 2                     | Reiss. 1.0   | 618         | 350      | 30          | 227         | 6            | 5        | 0         | 1    | 2    | 0    |
| 3                     | Yegaer 1.0   | 602         | 250      | 30          | 284         | 37           | 1        | 0         | 1    | 0    | 2    |
| 4                     | Reiner 1.0   | 540         | 300      | 0           | 202         | 9            | 29       | 0         | 1    | 1    | 2    |

| Results for 10 rounds |              |             |          |             |             |              |          |           |      |      |      |
|-----------------------|--------------|-------------|----------|-------------|-------------|--------------|----------|-----------|------|------|------|
| Rank                  | Name         | Total Score | Survival | Surv. Bonus | Bullet Dmg. | Bullet Bonus | Ram Dmg. | Ram Bonus | 1sts | 2nds | 3rds |
| 1                     | Reiner 1.0   | 839         | 500      | 60          | 206         | 9            | 47       | 17        | 1    | 3    | 1    |
| 2                     | Ackerman 1.0 | 817         | 300      | 30          | 438         | 47           | 2        | 0         | 2    | 0    | 3    |
| 3                     | Yegaer 1.0   | 699         | 250      | 30          | 396         | 22           | 1        | 0         | 2    | 0    | 1    |
| 4                     | Reiss. 1.0   | 370         | 150      | 0           | 218         | 0            | 1        | 0         | 0    | 2    | 1    |

## Daftar Pustaka

[what's the secret to making a good robot in robocode](#) (diakses pada 22 Maret 2025, pukul 01.42 WIB)

[Making a Walls bot not hit the walls](#) (diakses pada 22 Maret 2025, pukul 03.51 WIB)